Problem Statement: Use libraries like Matplotlib, Seaborn, and Plotly to create static **and** interactive visualisations

#(Matplotlib : Line + Bar Chart)

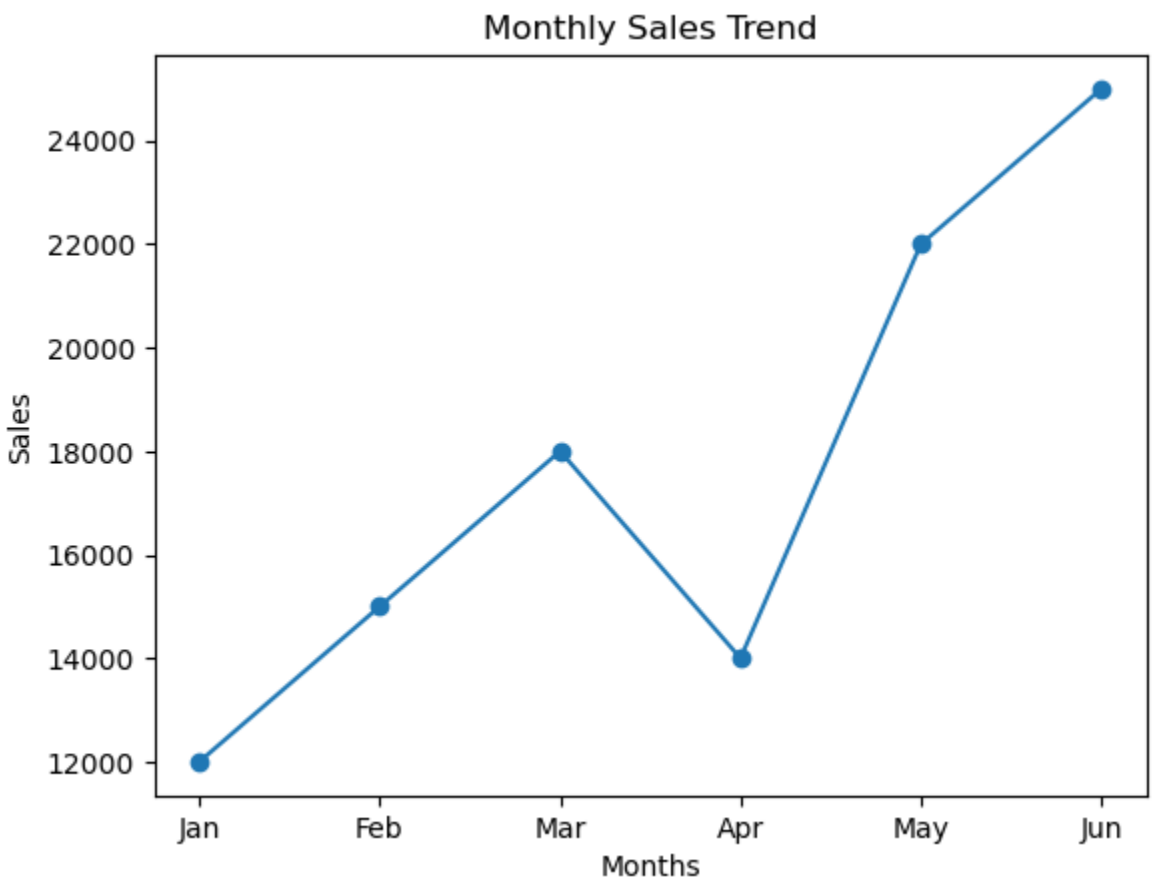
Data:
Month, Sales, Profit, Expenses
Jan, 12000, 2000, 8000
Feb, 15000, 2500, 9000
Mar, 18000, 3000, 10000
Apr, 14000, 1800, 8500
May, 22000, 4000, 12000
Jun, 25000, 5000, 13000

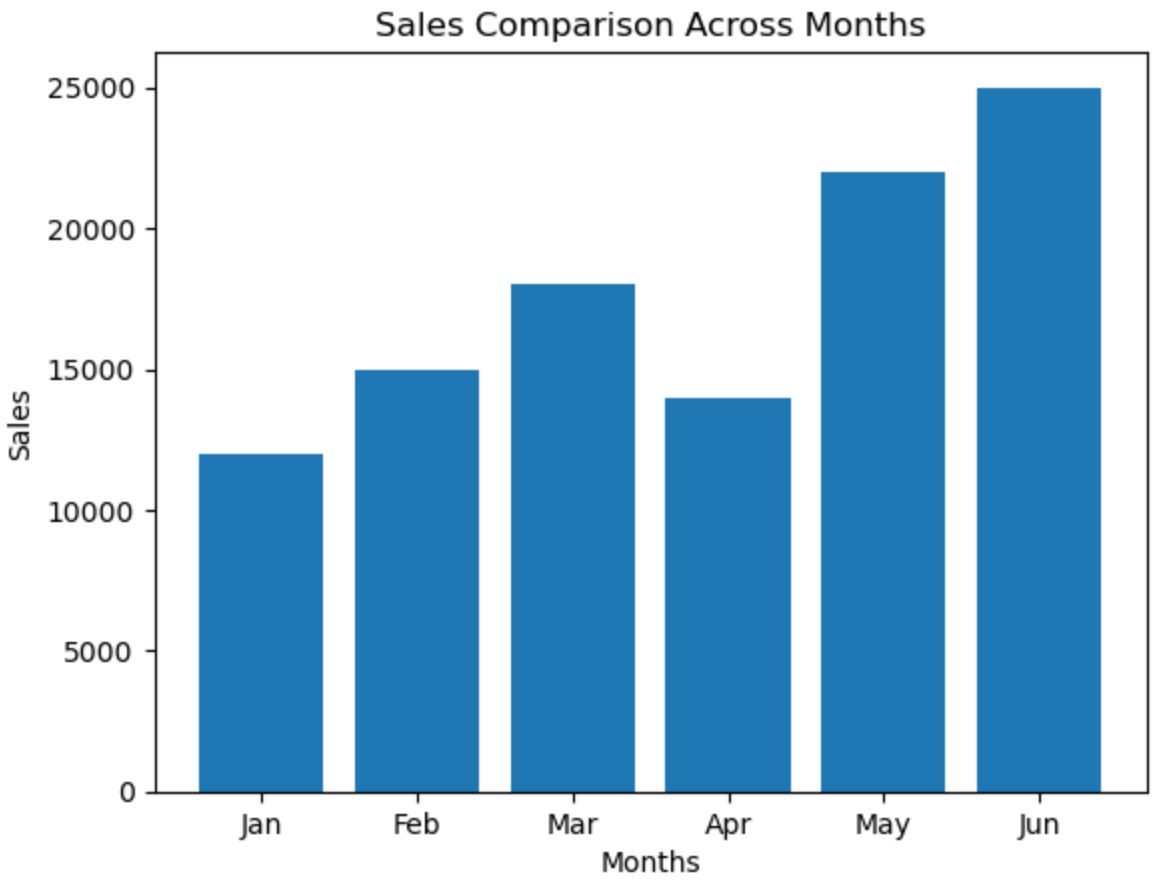
Tasks: (Matplotlib : Line **+** Bar Chart)
1. Plot a line chart to show monthly sales trend**.**
2. Plot a bar chart to compare sales across months**.**
3. Plot relationship between Sales **and** Profit**.**
4. Add X **and** y axis label
5. Add Bar Title
6. Add ticks

In [1]:

```
import matplotlib.pyplot as plt

# Data
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun
sales = [12000, 15000, 18000, 14000, 22000, 25000]
profit = [2000, 2500, 3000, 1800, 4000, 5000]
expenses = [8000, 9000, 10000, 8500, 12000, 13000]

# 1. Line Chart for Monthly Sales Trend
plt.figure()
plt.plot(months, sales, marker='o')
plt.xlabel("Months")                # X-axis label
plt.ylabel("Sales")                 # Y-axis label
plt.title("Monthly Sales Trend")    # Title
plt.xticks(months)                  # Add ticks
plt.show()

# 2. Bar Chart for Sales Comparison
plt.figure()
plt.bar(months, sales)
```

```
plt.xlabel("Months")
plt.ylabel("Sales")
plt.title("Sales Comparison Across Months")
plt.xticks(months)
plt.show()

# 3. Relationship between Sales and Profit (Scatter Plot)
plt.figure()
plt.scatter(sales, profit)
plt.xlabel("Sales")
plt.ylabel("Profit")
plt.title("Sales vs Profit Relationship")
plt.show()
```




Sales Comparison Across Months
25000
20000
15000
10000
5000
0
Sales
Jan
Feb
Mar
Apr
May
Jun
Months

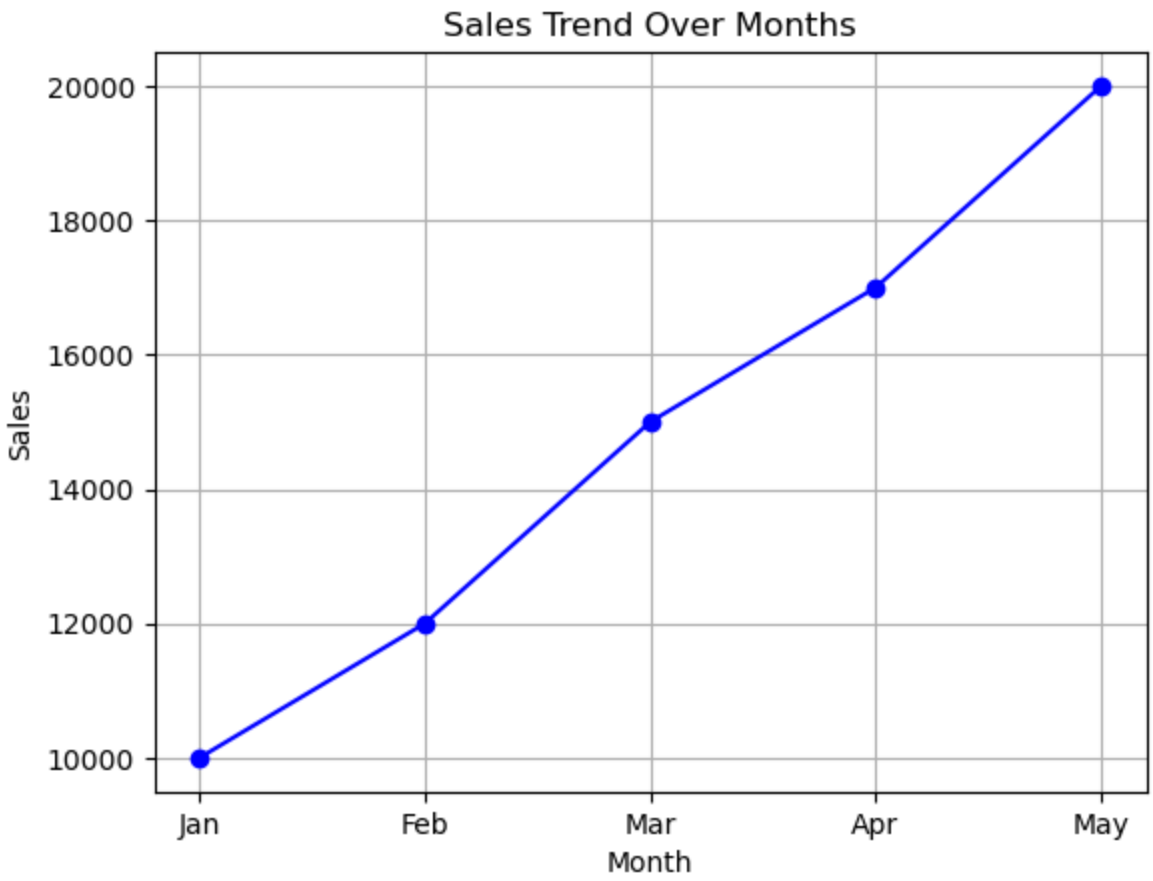
In [ ]:

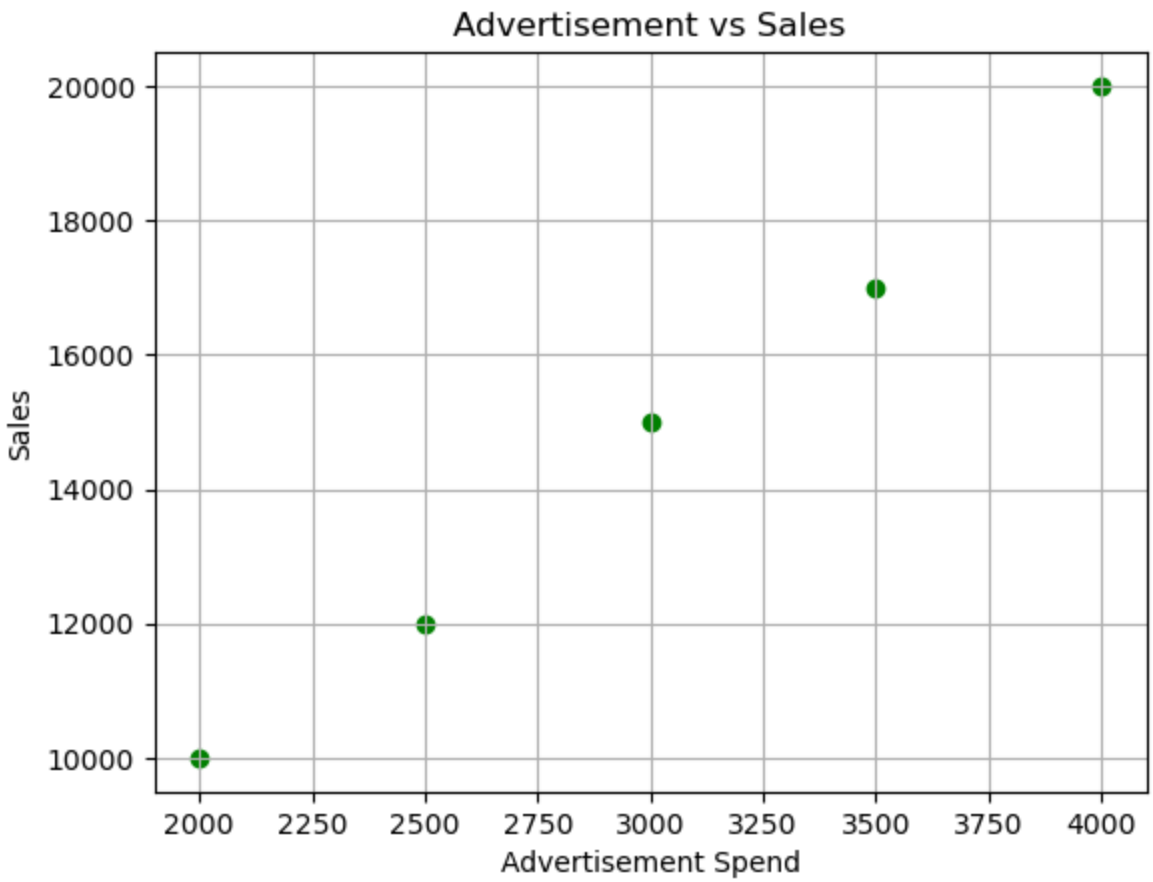
#(Matplotlib : Line + Scatter Chart)
Data:
Month, Sales, Advertisement
Jan, 10000, 2000
Feb, 12000, 2500
Mar, 15000, 3000
Apr, 17000, 3500
May, 20000, 4000

Tasks: (Matplotlib : Line **+** Scatter Chart)
1. Plot a line chart **for** Sales trend**.**
2. Plot a scatter plot **for** Advertisement vs Sales**.**
3. Add axis labels **and** titles**.**
4. Add gridlines**.**
5. Add X **and** y axis label
6. Add Bar Title

In [2]:

```
import pandas as pd
import matplotlib.pyplot as plt

# Data
data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May"],
    "Sales": [10000, 12000, 15000, 17000, 20000],
    "Advertisement": [2000, 2500, 3000, 3500, 4000]
}

df = pd.DataFrame(data)

# 1. Line Chart - Sales Trend

plt.plot(df["Month"], df["Sales"], marker='o', color='blue')

plt.title("Sales Trend Over Months")   # Title
plt.xlabel("Month")                    # X-axis label
plt.ylabel("Sales")                    # Y-axis label
plt.grid(True)

plt.show()

# 2. Scatter Plot - Advertisement vs Sales

plt.scatter(df["Advertisement"], df["Sales"], color='green')

plt.title("Advertisement vs Sales")   # Title
plt.xlabel("Advertisement Spend")     # X-axis label
plt.ylabel("Sales")                   # Y-axis label
plt.grid(True)

plt.show()
```


Sales Trend Over Months
20000
18000
16000
14000
12000
10000
Sales
Jan
Feb
Mar
Apr
May
Month



In [ ]:

#(Matplotlib : Line + Pie Chart)
Data:
Month, Expenses
Jan, 8000
Feb, 9000
Mar, 10000
Apr, 8500
May, 12000

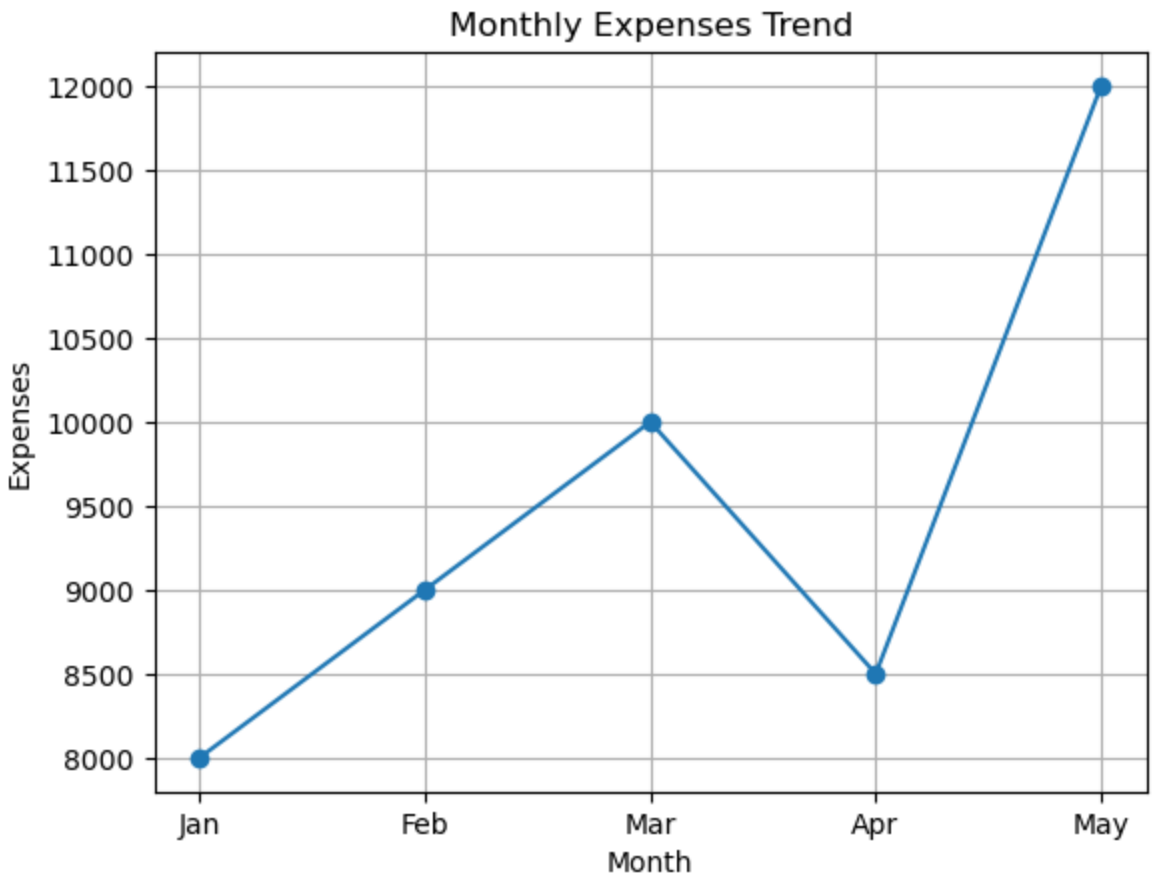
Category, Expense Share
Rent, 5000
Food, 3000
Transport, 1500
Others, 1000

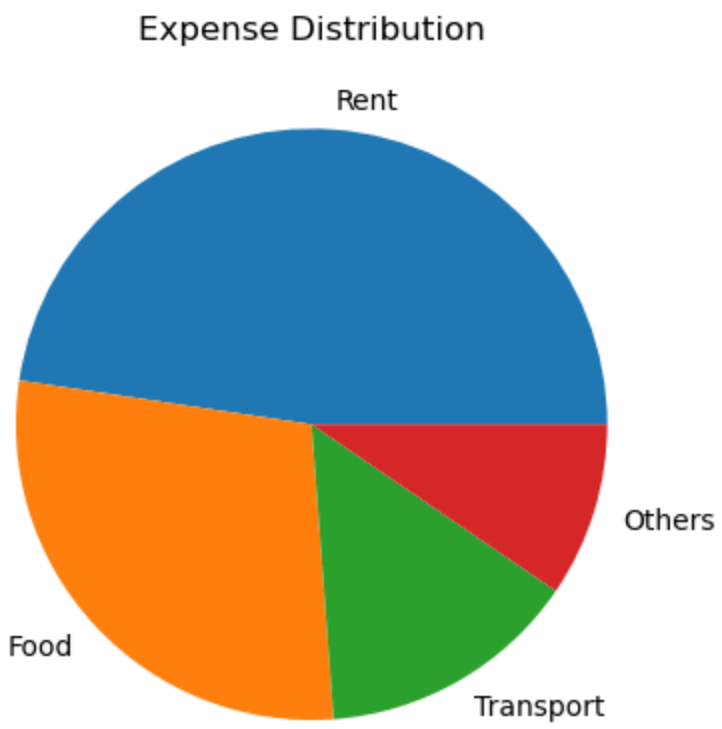
Tasks: (Matplotlib : Line **+** Pie Chart)
1. Plot a line chart **for** monthly expenses**.**

2. Plot a pie chart **for** expense distribution**.**
3. Add labels **and** titles**.**
4. Add gridlines to line chart**.**

In [4]:

```
import matplotlib.pyplot as plt

# Data
months = ["Jan", "Feb", "Mar", "Apr", "May"]
expenses = [8000, 9000, 10000, 8500, 12000]

categories = ["Rent", "Food", "Transport", "Others"]
expense_share = [5000, 3000, 1500, 1000]

#Line Chart for Monthly Expenses
plt.plot(months, expenses, marker='o')
plt.title("Monthly Expenses Trend")
plt.xlabel("Month")
plt.ylabel("Expenses")
plt.grid(True)
plt.show()

# Pie Chart for Expense Distribution
plt.pie(expense_share, labels=categories)
plt.title("Expense Distribution")
plt.show()
```


Monthly Expenses Trend
12000
11500
11000
10500
10000
9500
9000
8500
8000
Expenses
Jan
Feb
Mar
Apr
May
Month



In [ ]:

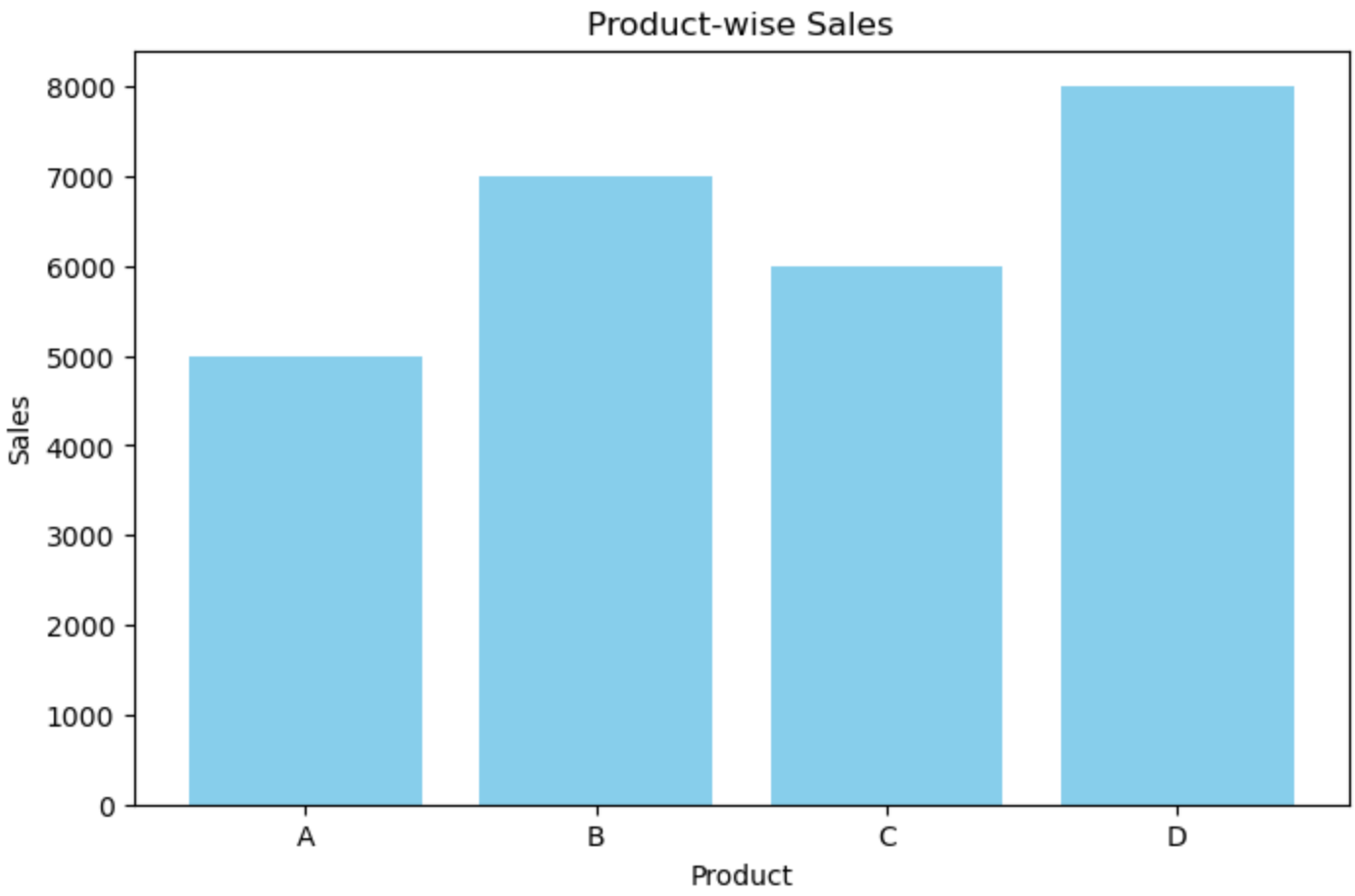
#(Matplotlib : Bar + Scatter Chart)
Data:
Product, Sales, Profit
A, 5000, 1000
B, 7000, 1500
C, 6000, 1200
D, 8000, 2000

Tasks: (Matplotlib : Bar **+** Scatter Chart)

1. Plot a bar chart **for** Sales**.**
2. Plot a scatter plot **for** Sales vs Profit**.**
3. Add axis labels**.**
4. Add title**.**
5. Add ticks**.**

In [5]:

```
import pandas as pd
import matplotlib.pyplot as plt

# Datat
data = {
    "Product": ["A", "B", "C", "D"],
    "Sales": [5000, 7000, 6000, 8000],
    "Profit": [1000, 1500, 1200, 2000]
}

df = pd.DataFrame(data)

# 1. Bar Chart - Sales

plt.figure(figsize=(8,5))
plt.bar(df["Product"], df["Sales"], color="skyblue")

plt.title("Product-wise Sales")   # Title
plt.xlabel("Product")            # X-axis label
plt.ylabel("Sales")              # Y-axis label

plt.xticks(df["Product"])  # Ticks

plt.show()

# 2. Scatter Plot - Sales vs Profit

plt.scatter(df["Sales"], df["Profit"], color="green")

plt.title("Sales vs Profit Relationship")  # Title
plt.xlabel("Sales")                        # X-axis label
plt.ylabel("Profit")                       # Y-axis label

plt.xticks(df["Sales"]) # Ticks
plt.yticks(df["Profit"])

plt.grid(True) #grid

plt.show()
```


Product-wise Sales
8000
7000
6000
5000
4000
3000
2000
1000
0
Sales
A
B
C
D
Product

In [ ]:

#(Seaborn: Line + Bar Chart)
Data:
Month, Sales, Profit
Jan, 12000, 2000
Feb, 15000, 2500
Mar, 18000, 3000
Apr, 14000, 1800
May, 22000, 4000
Jun, 25000, 5000

Tasks: (Seaborn: Line **+** Bar Chart)

1. Plot a line chart **for** Sales trend**.**
2. Plot a bar chart **for** Profit**.**
3. Add axis labels **and** title**.**

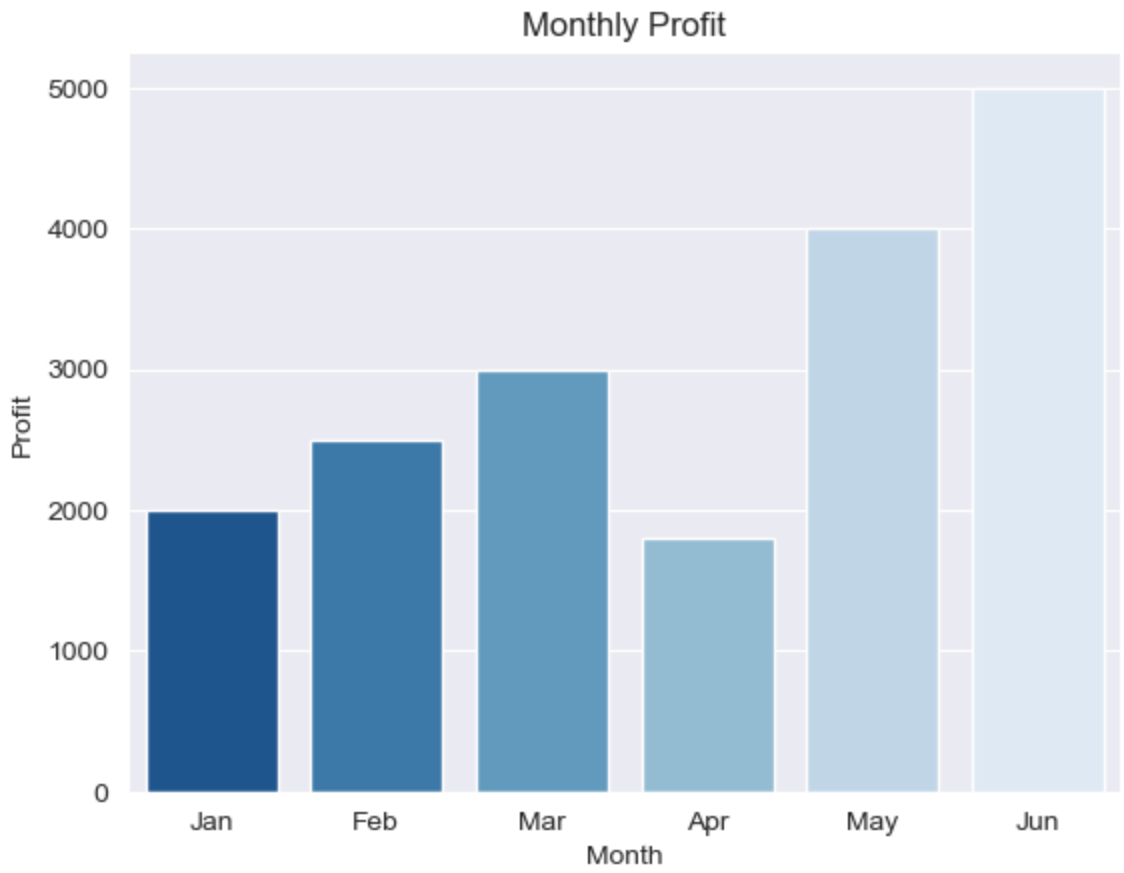
In [9]:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Dataset
data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Sales": [12000, 15000, 18000, 14000, 22000, 25000],
    "Profit": [2000, 2500, 3000, 1800, 4000, 5000]
}

df = pd.DataFrame(data)

# 1. Line Chart - Sales Trend

sns.lineplot(x="Month", y="Sales", data=df, marker="o", color="blue")

plt.title("Sales Trend Over Months")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.show()

# 2. Bar Chart - Profit

sns.barplot(x="Month", y="Profit", data=df, palette="Blues_r")

plt.title("Monthly Profit")
plt.xlabel("Month")
plt.ylabel("Profit")

plt.show()
```

C:\Users\Poonam\AppData\Local\Temp\ipykernel_9608\2983003429.py:26: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x="Month", y="Profit", data=df, palette="Blues_r")
```



In [ ]:

#(Seaborn: Scatter + Distribution)

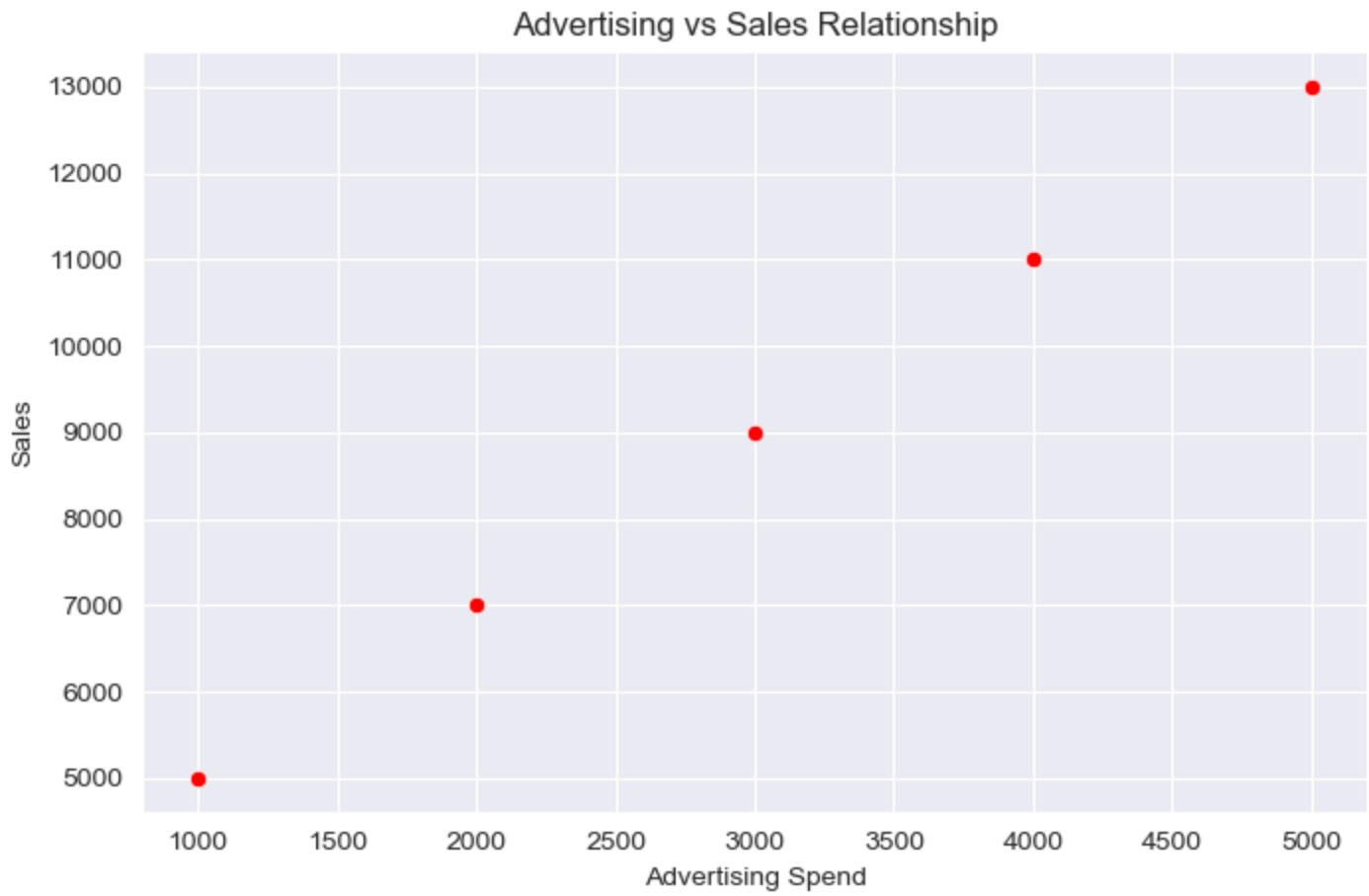
Data:
Advertising, Sales
  1000, 5000
  2000, 7000
  3000, 9000
  4000, 11000
  5000, 13000

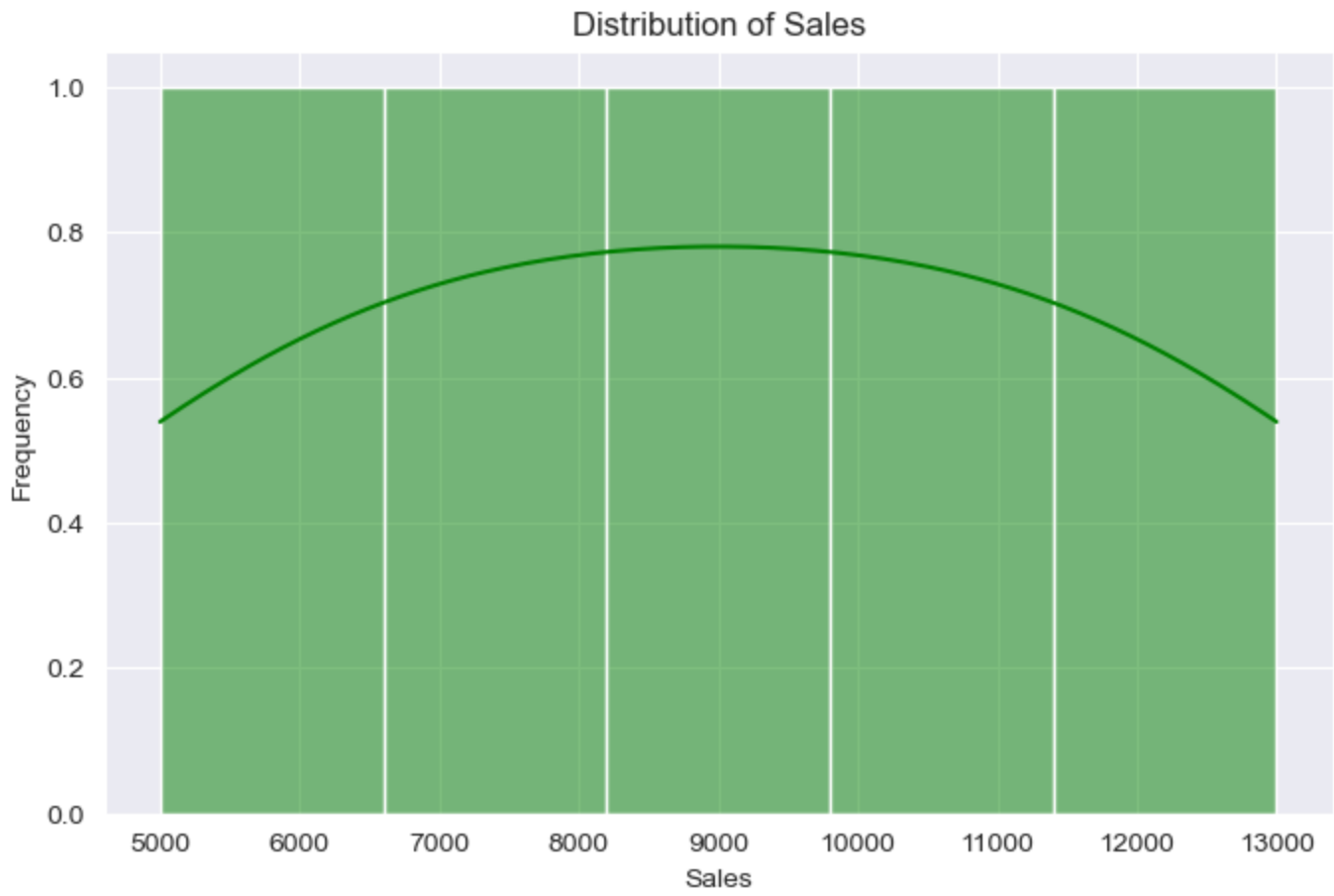
Tasks: (Seaborn: Scatter **+** Distribution)

Plot a scatter plot using Seaborn to show relationship**.**
Add regression line (regplot)**.**
Plot a distribution plot (histogram) of Sales**.**
Add labels **and** title**.**

In [13]:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Dataset

data = {
    "Advertising": [1000, 2000, 3000, 4000, 5000],
    "Sales": [5000, 7000, 9000, 11000, 13000]

}

df = pd.DataFrame(data)

# Set Seaborn style
sns.set_style("darkgrid")

# 1. Scatter Plot

plt.figure(figsize=(8,5))
sns.scatterplot(x="Advertising", y="Sales", data=df, color="red")

plt.title("Advertising vs Sales Relationship")
plt.xlabel("Advertising Spend")
plt.ylabel("Sales")

plt.show()


# 2. Distribution Plot (Histogram of Sales)

plt.figure(figsize=(8,5))
sns.histplot(df["Sales"], bins=5, kde=True, color="green")

plt.title("Distribution of Sales")
plt.xlabel("Sales")
plt.ylabel("Frequency")

plt.show()
```


Advertising vs Sales Relationship
Sales
13000
12000
11000
10000
9000
8000
7000
6000
5000
1000
1500
2000
2500
3000
3500
4000
4500
5000
Advertising Spend



In [ ]:

#(Seaborn : Line + Bar)

Data:
Month, Sales, Profit, Expenses
Jan, 12000, 2000, 8000
Feb, 15000, 2500, 9000
Mar, 18000, 3000, 10000
Apr, 14000, 1800, 8500
May, 22000, 4000, 12000
Jun, 25000, 5000, 13000

Tasks:(Seaborn : Line **+** Bar)

1. Plot a line chart **for** Sales trend**.**
2. Plot a bar chart **for** Expenses**.**
3. Add axis labels **and** title**.**

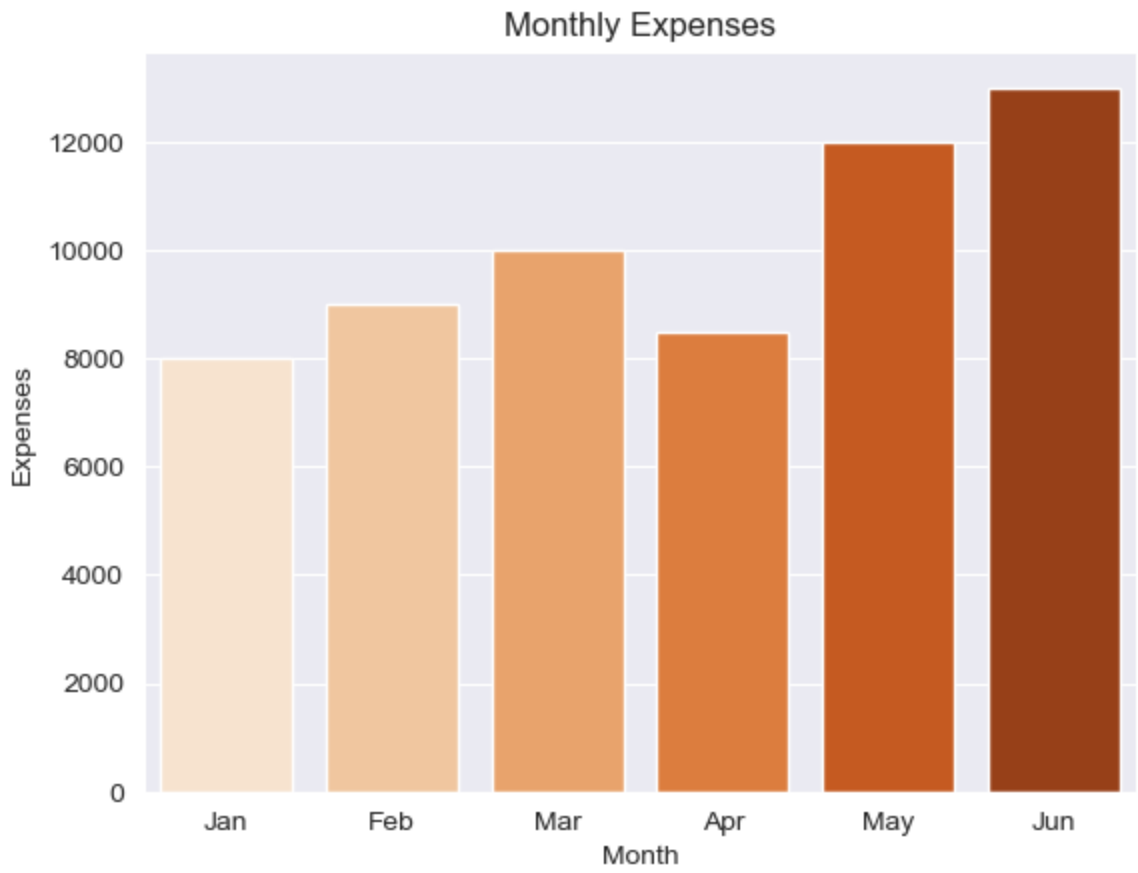
In [14]:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt


# Dataset
data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Sales": [12000, 15000, 18000, 14000, 22000, 25000],
    "Profit": [2000, 2500, 3000, 1800, 4000, 5000],
    "Expenses": [8000, 9000, 10000, 8500, 12000, 13000]
}

df = pd.DataFrame(data)

# 1. Line Chart - Sales Trend

sns.lineplot(x="Month", y="Sales", data=df, marker="o", color="blue")

plt.title("Sales Trend Over Months")
plt.xlabel("Month")
plt.ylabel("Sales")

plt.show()

# 2. Bar Chart - Expenses

sns.barplot(x="Month", y="Expenses", data=df, palette="Oranges")

plt.title("Monthly Expenses")
plt.xlabel("Month")
plt.ylabel("Expenses")

plt.show()
```

C:\Users\Poonam\AppData\Local\Temp\ipykernel_9608\3961684904.py:30: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x="Month", y="Expenses", data=df, palette="Oranges")
```



In [ ]:

#(Plotly : Bar + Pie)

Data:
Category, Sales, Profit, Share
Electronics, 40000, 10000, 40
Clothing, 25000, 6000, 25
Groceries, 15000, 3000, 15
Furniture, 20000, 5000, 20

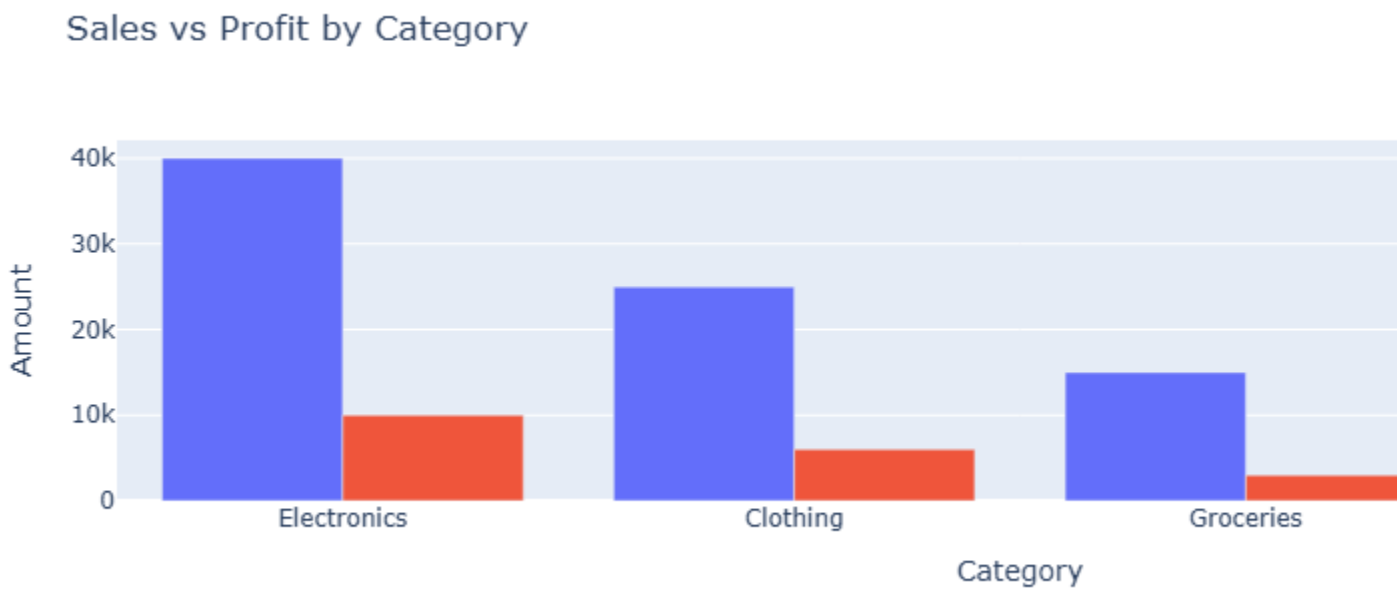
Tasks: (Plotly : Bar **+** Pie)

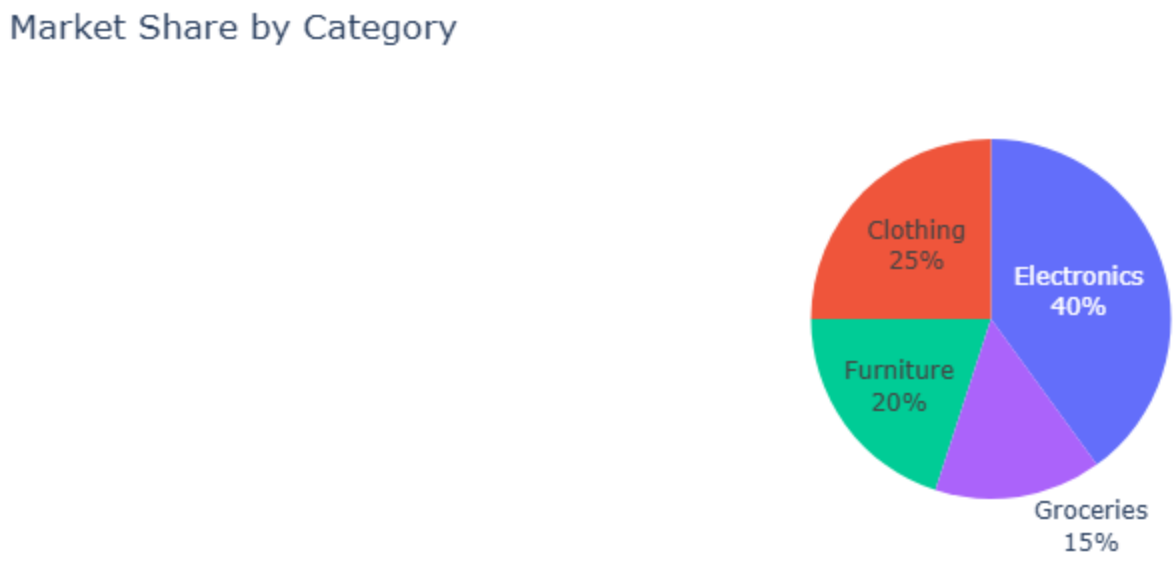
1. Create a bar chart **for** Sales vs Profit**.**
2. Create a pie chart using Share column**.**
3. Add labels **and** titles**.**
4. Show percentages **in** pie chart**.**
5. Use interactive hover features**.**
6. Customize colors**.**

In [15]:

```
import pandas as pd
import plotly.express as px


# Dataset

data = {
    "Category": ["Electronics", "Clothing", "Groceries", "Furniture"],
    "Sales": [40000, 25000, 15000, 20000],
    "Profit": [10000, 6000, 3000, 5000],
    "Share": [40, 25, 15, 20]
}

df = pd.DataFrame(data)


# 1. Bar Chart - Sales vs Profit

fig1 = px.bar(
    df,
    x="Category",
    y=["Sales", "Profit"],
    barmode="group",
    title="Sales vs Profit by Category",
    color_discrete_sequence=["#636EFA", "#EF553B"]  # customized colors
)

fig1.update_layout(
    xaxis_title="Category",
    yaxis_title="Amount"
)

fig1.show()


# 2. Pie Chart - Share Distribution

fig2 = px.pie(
    df,
    names="Category",
    values="Share",
    title="Market Share by Category",
    color_discrete_sequence=["#636EFA", "#EF553B", "#00CC96", "#AB63FA"],
```

```
    hole=0  # set to 0.4 for donut chart if needed
)

fig2.update_traces(textinfo="percent+label")

fig2.show()
```





In [ ]:

#(Seaborn – Boxplot + Violin with Multiple Columns)

Data:
Department, Salary, Experience, Bonus
HR, 30000, 2, 2000
HR, 32000, 3, 2500
IT, 50000, 5, 5000
IT, 55000, 6, 6000
Finance, 45000, 4, 4000
Finance, 47000, 5, 4500

Tasks: (Seaborn – Boxplot **+** Violin **with** Multiple Columns)

1. Plot a boxplot of Salary by Department**.**
2. Plot a violin plot **for** Salary distribution**.**
3. Add labels **and** title**.**

In [17]:

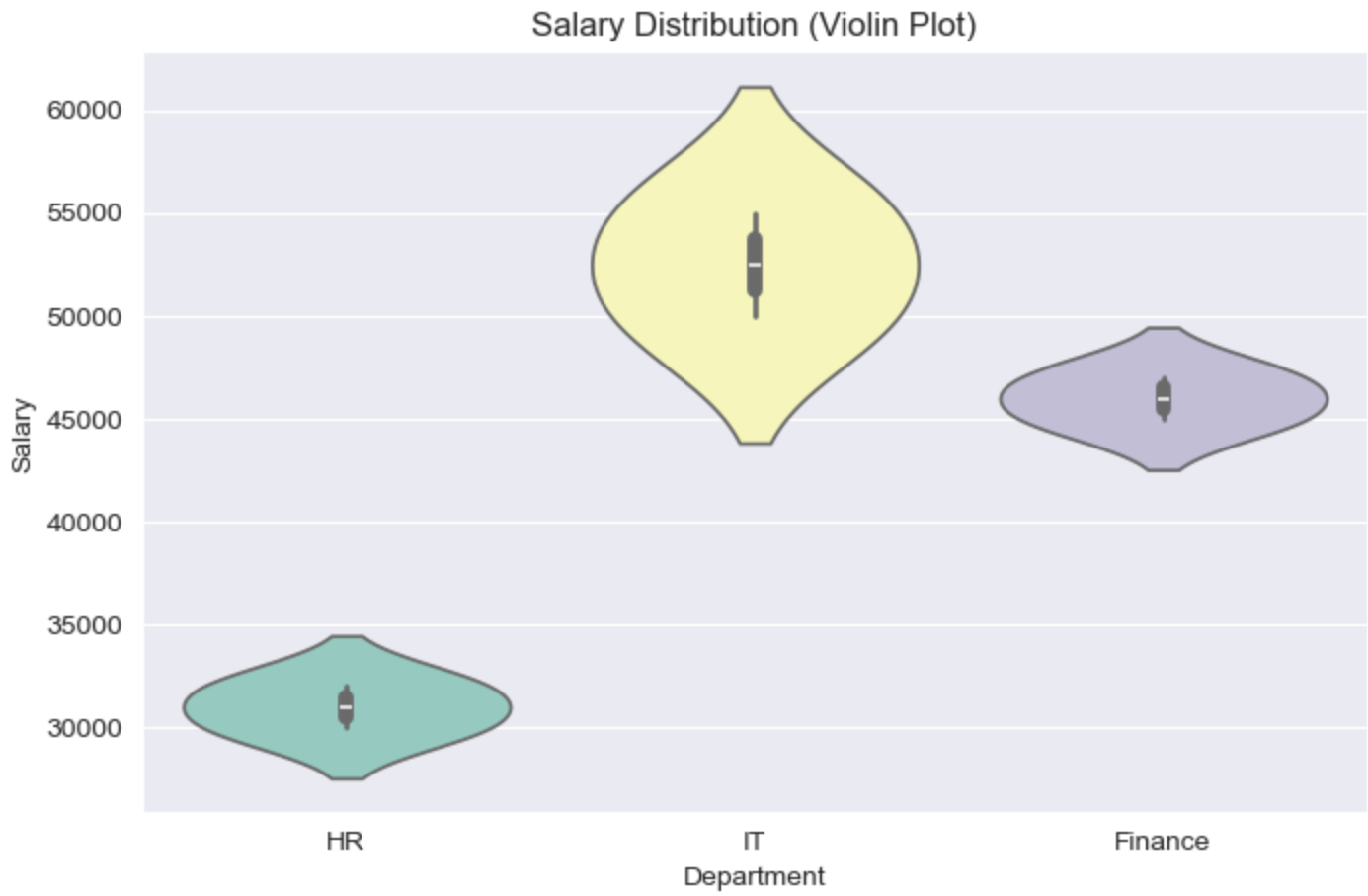
```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Dataset

data = {
    "Department": ["HR", "HR", "IT", "IT", "Finance", "Finance"],
    "Salary": [30000, 32000, 50000, 55000, 45000, 47000],
    "Experience": [2, 3, 5, 6, 4, 5],
    "Bonus": [2000, 2500, 5000, 6000, 4000, 4500]
}

df = pd.DataFrame(data)

# 1. Boxplot - Salary by Department

sns.boxplot(x="Department", y="Salary", data=df, palette="Set2")

plt.title("Salary Distribution by Department")
plt.xlabel("Department")
plt.ylabel("Salary")

plt.show()

# 2. Violin Plot - Salary Distribution

sns.violinplot(x="Department", y="Salary", data=df, palette="Set3")

plt.title("Salary Distribution (Violin Plot)")
plt.xlabel("Department")
plt.ylabel("Salary")

plt.show()
```

```
C:\Users\Poonam\AppData\Local\Temp\ipykernel_9608\2730387575.py:22: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign
the `x` variable to `hue` and set `legend=False` for the same effect.

  sns.boxplot(x="Department", y="Salary", data=df, palette="Set2")
```

C:\Users\Poonam\AppData\Local\Temp\ipykernel_9608\2730387575.py:33: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.violinplot(x="Department", y="Salary", data=df, palette="Set3")
```



In [ ]:

#(Seaborn – Boxplot + Violin with Multiple Columns)

Data:
Department, Salary, Experience, Bonus
HR, 30000, 2, 2000
HR, 32000, 3, 2500
IT, 50000, 5, 5000
IT, 55000, 6, 6000
Finance, 45000, 4, 4000
Finance, 47000, 5, 4500

Tasks: (Seaborn – Boxplot **+** Violin **with** Multiple Columns)

1. Plot a boxplot of Salary by Department**.**
2. Plot a violin plot **for** Salary distribution**.**
3. Analyze relationship between Experience **and** Salary**.**

4. Add labels **and** title**.**
5. Identify variation across departments**.**

In [18]:

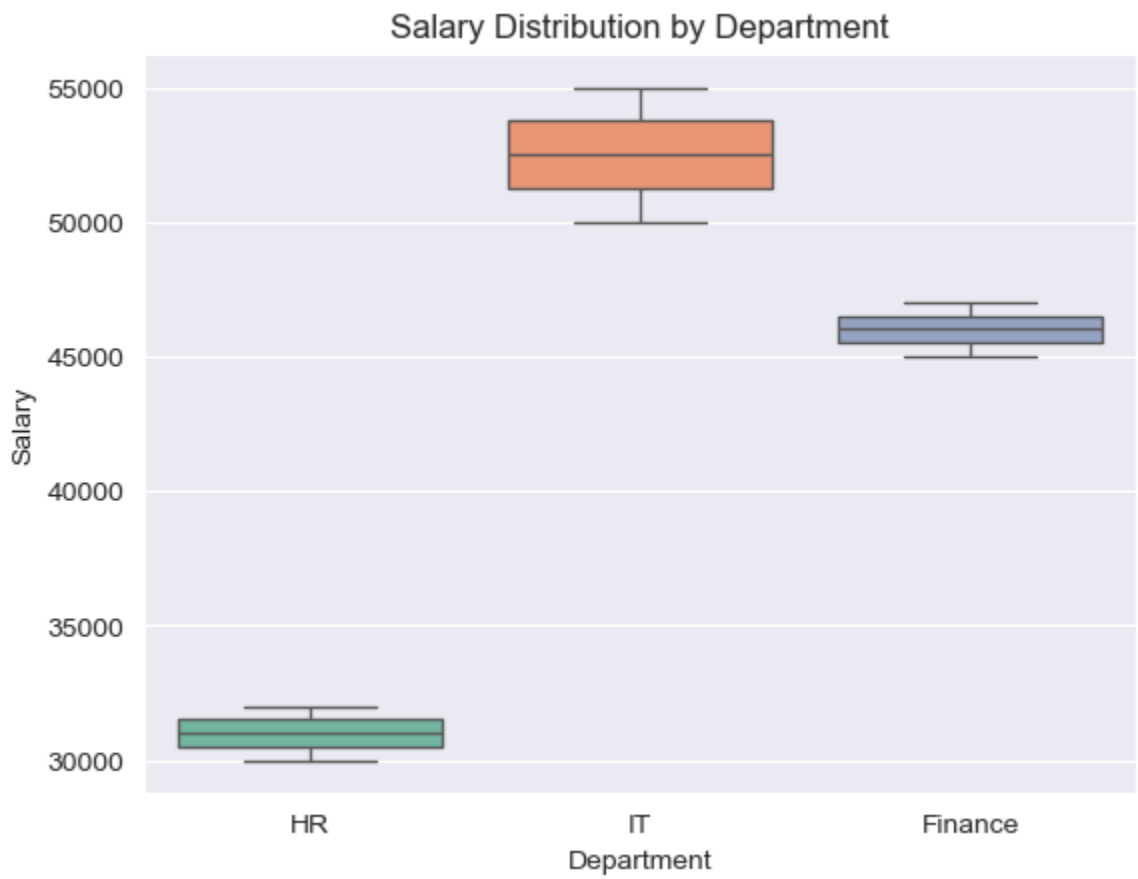
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Dataset

data = {
    "Department": ["HR", "HR", "IT", "IT", "Finance", "Finance"],
    "Salary": [30000, 32000, 50000, 55000, 45000, 47000],
    "Experience": [2, 3, 5, 6, 4, 5],
    "Bonus": [2000, 2500, 5000, 6000, 4000, 4500]
}

df = pd.DataFrame(data)

# 1. Boxplot - Salary by Department

sns.boxplot(x="Department", y="Salary", data=df, palette="Set2")

plt.title("Salary Distribution by Department")
plt.xlabel("Department")
plt.ylabel("Salary")

plt.show()

# 2. Violin Plot - Salary Distribution

sns.violinplot(x="Department", y="Salary", data=df, palette="Set3")

plt.title("Salary Distribution (Violin Plot)")
plt.xlabel("Department")
plt.ylabel("Salary")

plt.show()

# 3. Relationship - Experience vs Salary

sns.scatterplot(x="Experience", y="Salary", hue="Department", data=df, s=100)

plt.title("Experience vs Salary Relationship")
plt.xlabel("Experience (Years)")
```

```
plt.ylabel("Salary")

plt.show()
```

C:\Users\Poonam\AppData\Local\Temp\ipykernel_9608\3280476768.py:18: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

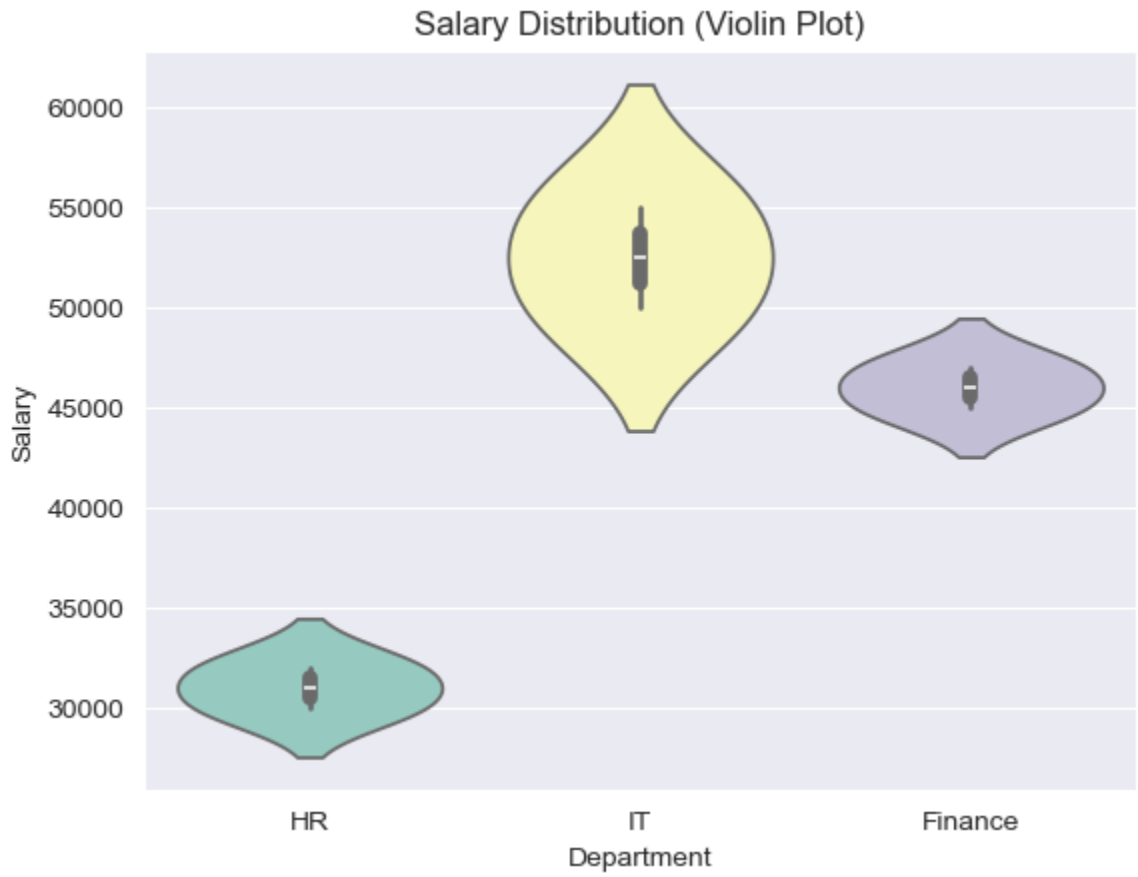
```
sns.boxplot(x="Department", y="Salary", data=df, palette="Set2")
```



C:\Users\Poonam\AppData\Local\Temp\ipykernel_9608\3280476768.py:28: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.violinplot(x="Department", y="Salary", data=df, palette="Set3")
```


Salary Distribution (Violin Plot)
60000
55000
50000
45000
40000
35000
30000
Salary
HR
IT
Finance
Department

In [ ]:

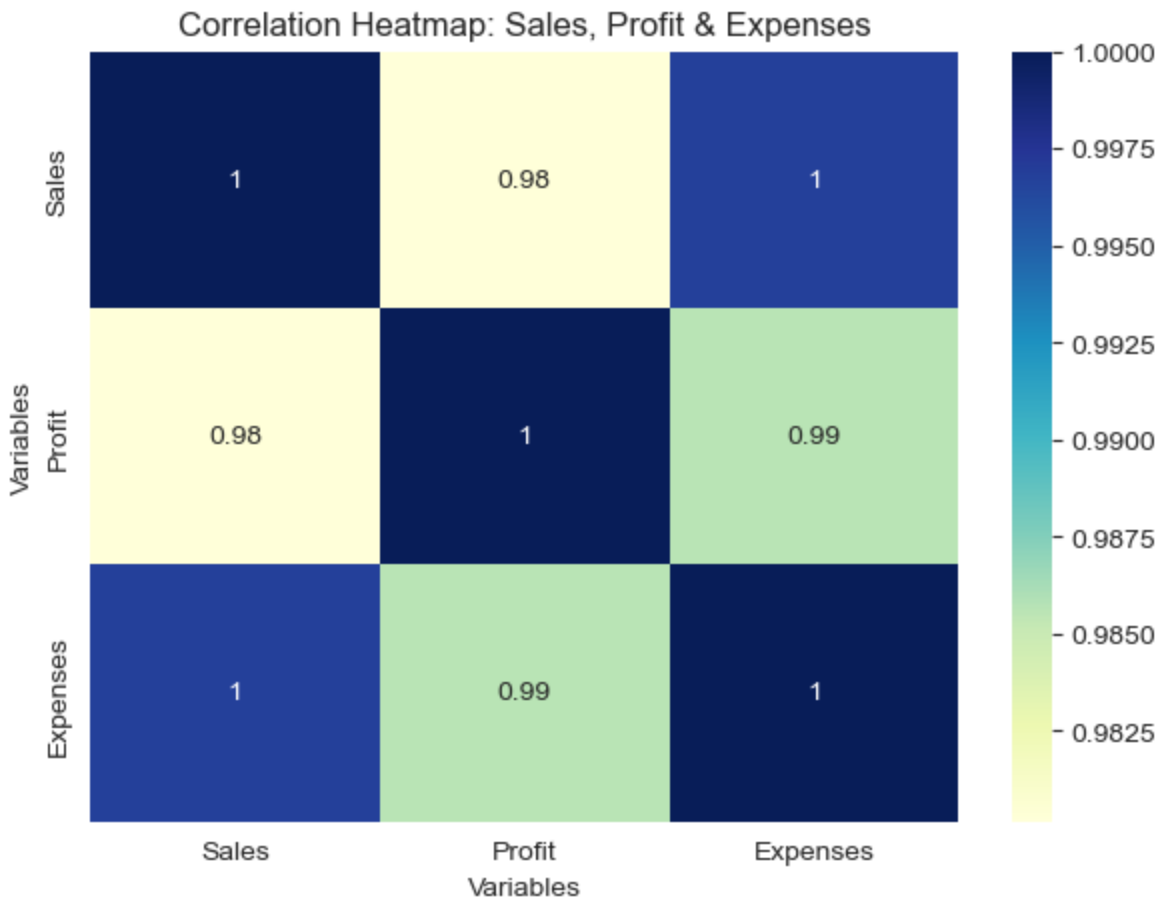
(Seaborn – Heatmap **with** Correlation)

Data:
Month, Sales, Profit, Expenses
Jan, 12000, 2000, 8000
Feb, 15000, 2500, 9000
Mar, 18000, 3000, 10000
Apr, 14000, 1800, 8500
May, 22000, 4000, 12000
Jun, 25000, 5000, 13000

Tasks:
1. Create a correlation matrix using pandas**.**
2. Plot a heatmap using Seaborn**.**
3. Add title **and** labels**.**
4. Apply a suitable color map**.**

In [19]:

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Dataset

data = {
    "Month": ["Jan", "Feb", "Mar", "Apr", "May", "Jun"],
    "Sales": [12000, 15000, 18000, 14000, 22000, 25000],
    "Profit": [2000, 2500, 3000, 1800, 4000, 5000],
    "Expenses": [8000, 9000, 10000, 8500, 12000, 13000]
}

df = pd.DataFrame(data)


# Heatmap

plt.figure(figsize=(7,5))
sns.set_style("darkgrid")

sns.heatmap(
    df[["Sales", "Profit", "Expenses"]].corr(),
    annot=True,
    cmap="YlGnBu"
)

plt.title("Correlation Heatmap: Sales, Profit & Expenses")
plt.xlabel("Variables")
plt.ylabel("Variables")

plt.show()
```



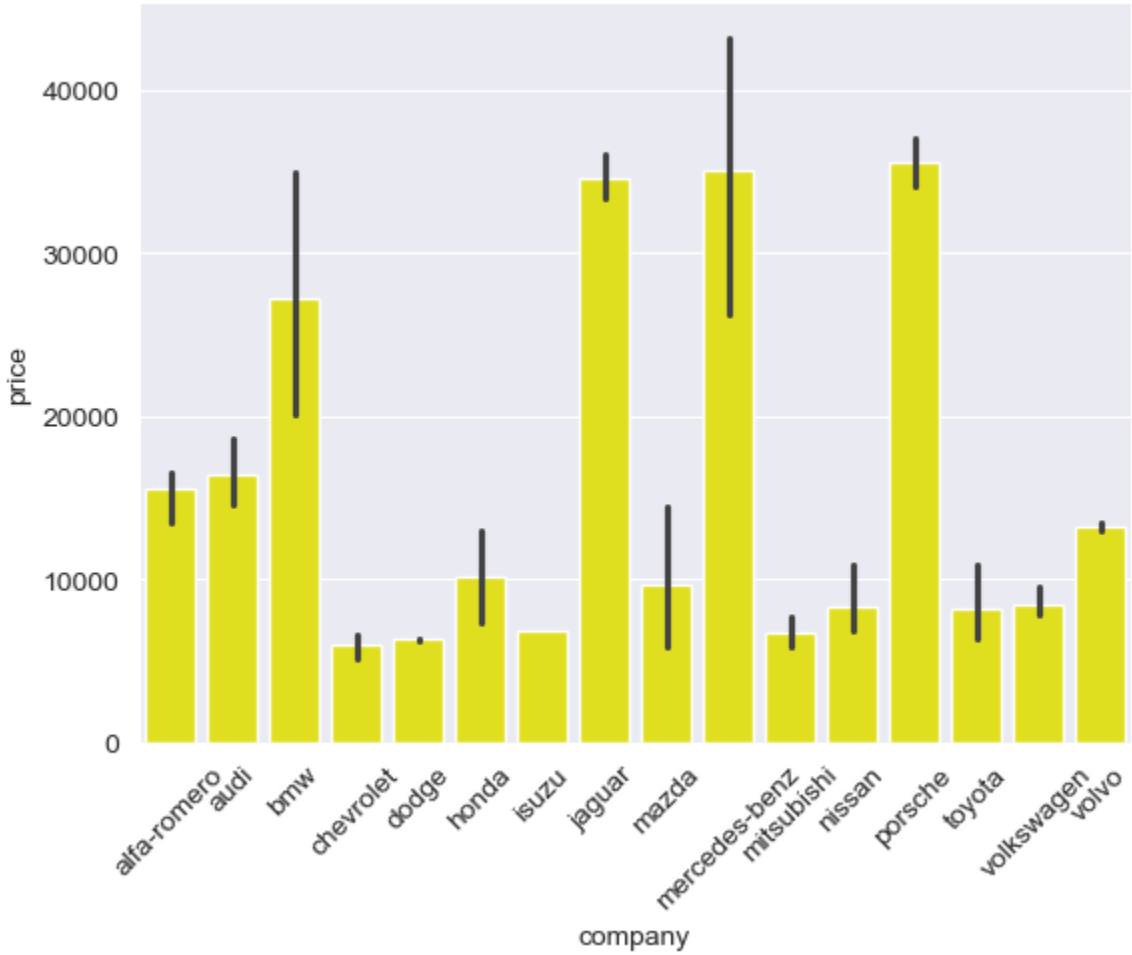
In [25]:

```
#load dataset using the Pandas read_csv() function.
#D:\1_2025-26\Sem-II\AVT\PPT

import pandas as pd
autom_data=pd.read_csv("D:/1_2025-26/Sem-II/AVT/Lab/Automobile_Data.csv")
print(autom_data)

#Display First 10 Lines using function head()
autom_data.head()

#Display Last 10 Lines using function tail()
autom_data.tail()

#Display information about data using function info()
autom_data.info()
```

```
   index      company   body-style  wheel-base  length engine-type  \
0      0  alfa-romero  convertible        88.6   168.8        dohc
1      1  alfa-romero  convertible        88.6   168.8        dohc
2      2  alfa-romero    hatchback        94.5   171.2        ohcv
3      3         audi        sedan        99.8   176.6         ohc
4      4         audi        sedan        99.4   176.6         ohc
..   ...          ...          ...         ...     ...         ...
56    81   volkswagen        sedan        97.3   171.7         ohc
57    82   volkswagen        sedan        97.3   171.7         ohc
58    86   volkswagen        sedan        97.3   171.7         ohc
59    87        volvo        sedan       104.3   188.8         ohc
60    88        volvo        wagon       104.3   188.8         ohc

   num-of-cylinders  horsepower  average-mileage    price
0              four         111               21  13495.0
1              four         111               21  16500.0
2               six         154               19  16500.0
3              four         102               24  13950.0
4              five         115               18  17450.0
..              ...         ...              ...     ...
56              four          85               27   7975.0
57              four          52               37   7995.0
58              four         100               26   9995.0
59              four         114               23  12940.0
60              four         114               23  13415.0

[61 rows x 10 columns]
<class 'pandas.DataFrame'>
RangeIndex: 61 entries, 0 to 60
Data columns (total 10 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   index             61 non-null     int64
 1   company           61 non-null     str
 2   body-style        61 non-null     str
 3   wheel-base        61 non-null     float64
 4   length            61 non-null     float64
 5   engine-type       61 non-null     str
 6   num-of-cylinders  61 non-null     str
 7   horsepower        61 non-null     int64
 8   average-mileage   61 non-null     int64
 9   price             58 non-null     float64
dtypes: float64(3), int64(3), str(4)
memory usage: 6.1 KB
```

In [26]:

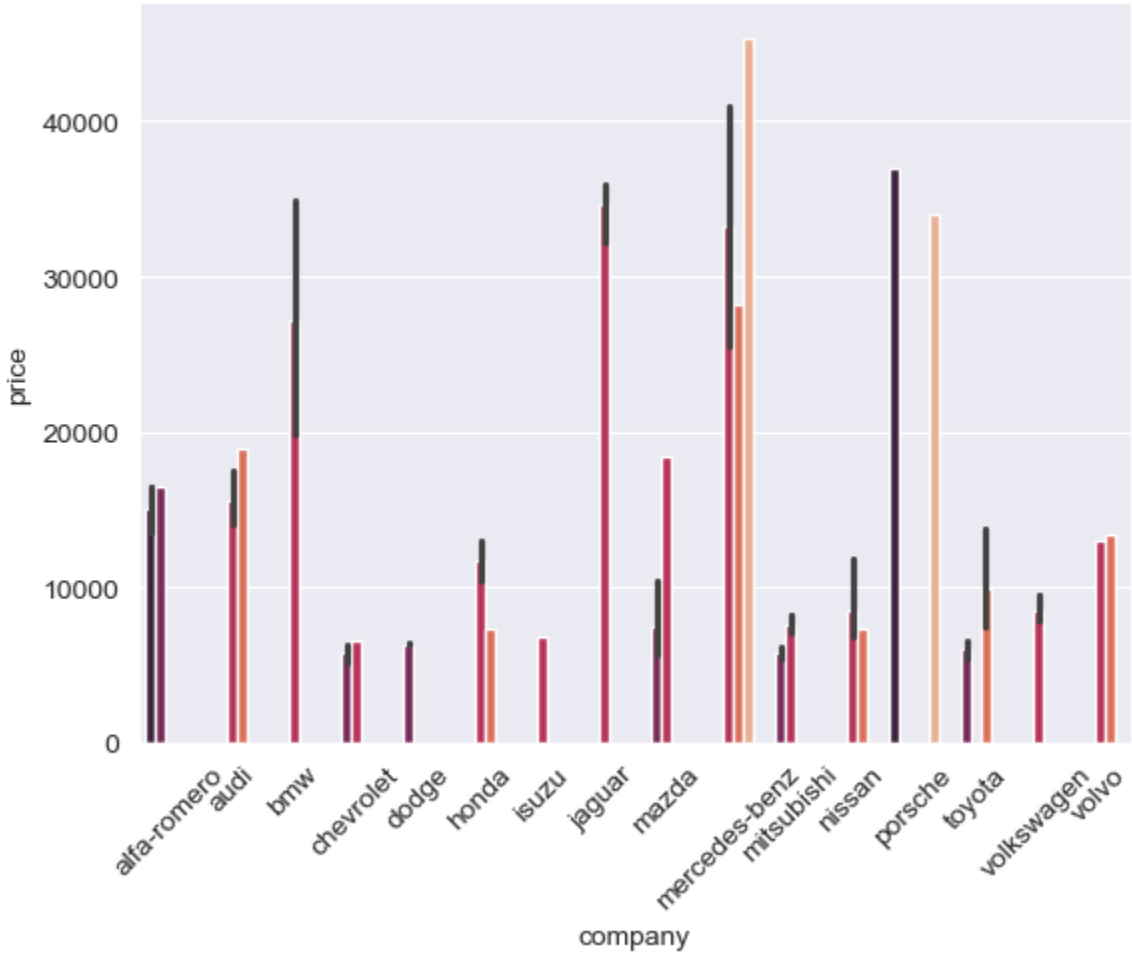
```
import seaborn as sns
import matplotlib.pyplot as plt
# Seaborn : Bar Graph
sns.barplot(x='company', y='price', color="yellow",data=autom_data)
plt.xticks(rotation=45)
plt.show()
```



In [27]:

#Seaborn library also has the palette attribute which you can use to give different colors to the bars.

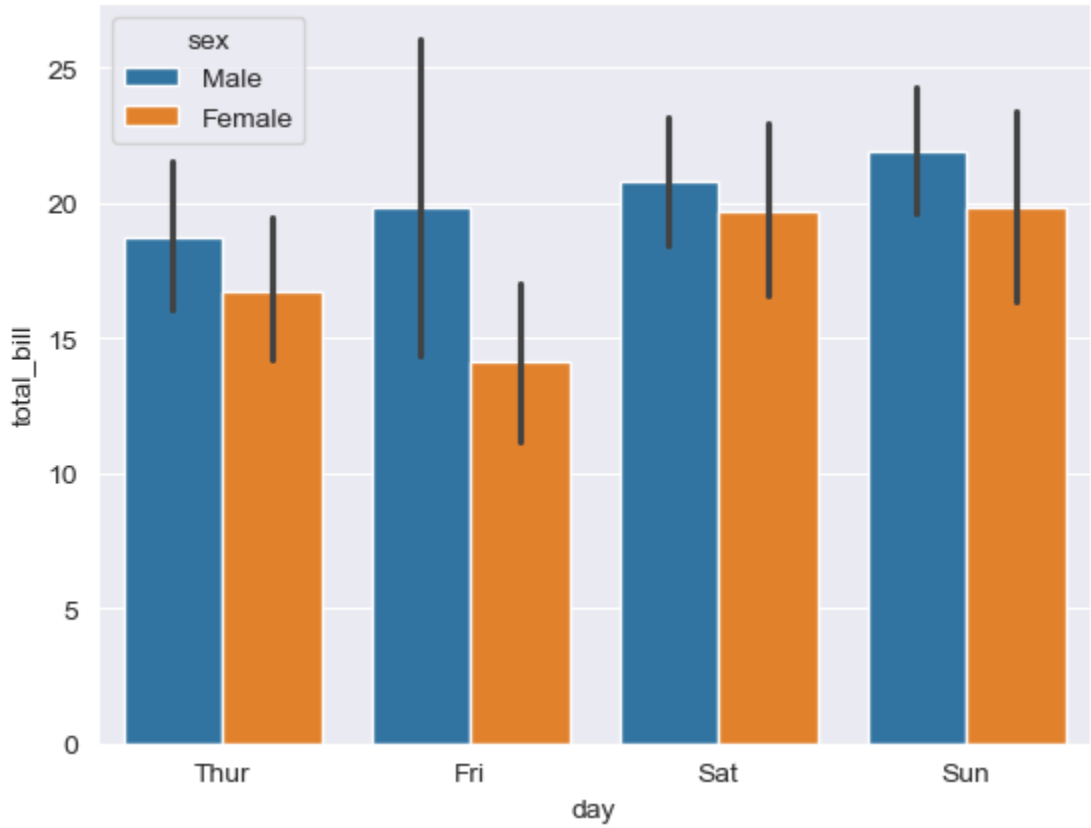
```
# Seaborn : Bar Graph
import seaborn as sns
import matplotlib.pyplot as plt

sns.barplot(x='company', y='price', palette="rocket", hue='body-style',legend=False,data=autom_data)
plt.xticks(rotation=45)
plt.show()
```



In [29]:

```
# Seaborn : Bar Graph
```

```
tips = sns.load_dataset('tips')
sns.barplot(x='day', y='total_bill', hue='sex', data=tips)
plt.show()
```



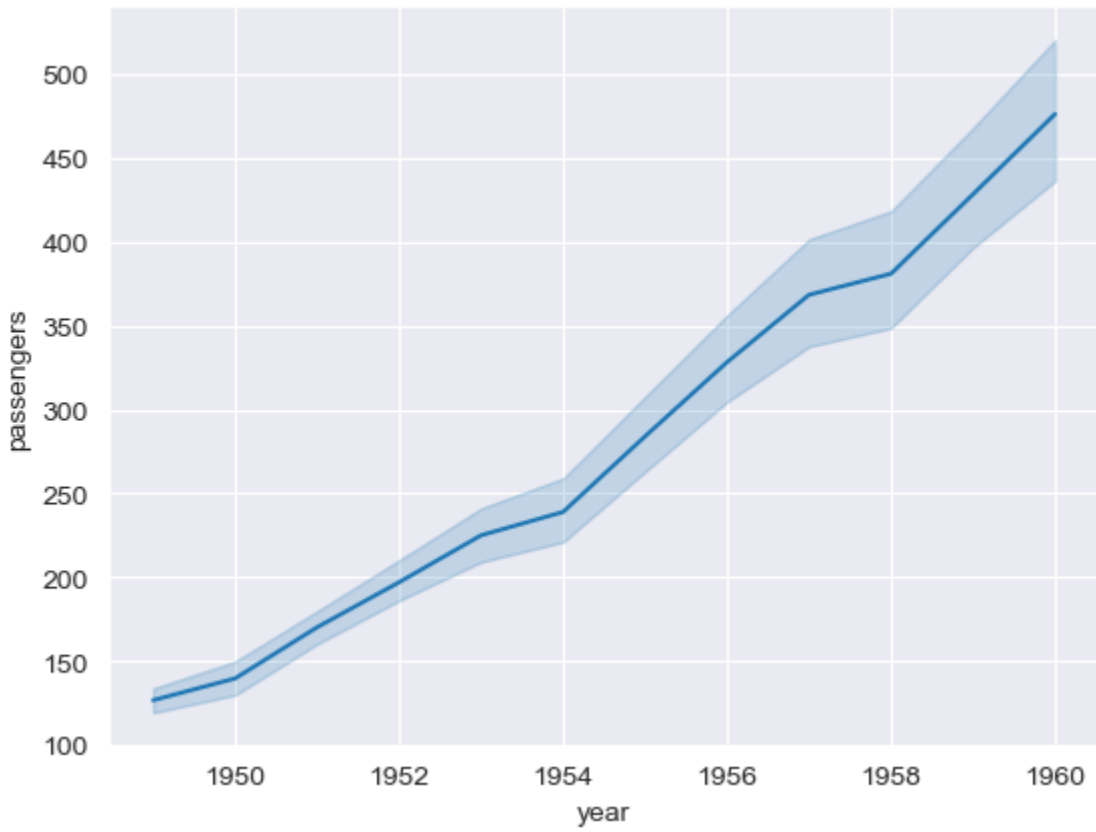
In [30]:

```
# LINE PLOT: Line plots show trends or changes over time or ordered categories by
connecting data points with lines.
# Seaborn : Line Graph
flights = sns.load_dataset("flights")
print(flights.head())

sns.lineplot(x="year", y="passengers", data=flights)
```

```
   year month  passengers
0  1949   Jan         112
1  1949   Feb         118
2  1949   Mar         132
3  1949   Apr         129
4  1949   May         121
```

Out[30]:

```
<Axes: xlabel='year', ylabel='passengers'>
```



In [ ]:

Streamlit Codes are given **in** sepearate files refer those files **for** Practice**.**
Follow the steps to execute Streamlit code

Steps to Run Streamlit Code
1. Install Streamlit (If it **is not** installed)

First, install Streamlit using pip:

pip install streamlit

2. Create a Python File

Create a new file, **for** example:

app.py

3. Save the File

Make sure your file **is** saved properly **as**:

app.py

4. Open Anaconda Prompt

Go to the folder where your file **is** saved:

Run Streamlit App
streamlit run app.py

5. Open **in** Browser

After running:

Streamlit will automatically open a tab **in** the Browser

In [31]:

```
#To perform data cleaning and preprocessing using pandas to prepare the data for
visualisation.
import pandas as pd
import seaborn as sns

# Load a built-in dataset from seaborn
data = sns.load_dataset("titanic")

# 1. Detect all missing values (True/False DataFrame)
print("data.isnull():")
print(data.isnull())

# 2. Detect all missing values (same as isnull)
print("\ndata.isna():")
```

```
print(data.isna())

# 3. Check if each column has any missing values
print("\ndata.isna().any():")
print(data.isna().any())

# 4. Count missing values in each column
print("\ndata.isna().sum():")
print(data.isna().sum())

# 5. Count number of columns with missing values
print("\ndata.isna().any().sum():")
print(data.isna().any().sum())

#6.fillna()
df_fill = data.fillna(10)
df_fill = data.fillna({"age": 10})
print("\nAfter fillna():\n", df_fill)

#7.dropna()
#print("\ndropna():")
#print(data.dropna())

#8.duplicated()
print("\nDuplicated Rows:")
print(data.duplicated())

#9.drop_duplicates()
cleaned_data = data.drop_duplicates()
print("\nDataFrame after drop_duplicates():\n", cleaned_data)
```

```
data.isnull():
    survived  pclass    sex    age  sibsp  parch   fare  embarked  class  \
0      False   False  False  False  False  False  False     False  False
1      False   False  False  False  False  False  False     False  False
2      False   False  False  False  False  False  False     False  False
3      False   False  False  False  False  False  False     False  False
4      False   False  False  False  False  False  False     False  False
..       ...     ...    ...    ...    ...    ...    ...       ...    ...
886    False   False  False  False  False  False  False     False  False
887    False   False  False  False  False  False  False     False  False
888    False   False  False   True  False  False  False     False  False
889    False   False  False  False  False  False  False     False  False
890    False   False  False  False  False  False  False     False  False

      who  adult_male   deck  embark_town  alive  alone
0    False       False   True        False  False  False
```

```
1    False      False  False       False  False  False
2    False      False   True       False  False  False
3    False      False  False       False  False  False
4    False      False   True       False  False  False
..     ...        ...    ...         ...    ...    ...
886  False      False   True       False  False  False
887  False      False  False       False  False  False
888  False      False   True       False  False  False
889  False      False  False       False  False  False
890  False      False   True       False  False  False

[891 rows x 15 columns]

data.isna():
    survived  pclass    sex    age  sibsp  parch   fare  embarked  class  \
0      False   False  False  False  False  False  False     False  False
1      False   False  False  False  False  False  False     False  False
2      False   False  False  False  False  False  False     False  False
3      False   False  False  False  False  False  False     False  False
4      False   False  False  False  False  False  False     False  False
..       ...     ...    ...    ...    ...    ...    ...       ...    ...
886    False   False  False  False  False  False  False     False  False
887    False   False  False  False  False  False  False     False  False
888    False   False  False   True  False  False  False     False  False
889    False   False  False  False  False  False  False     False  False
890    False   False  False  False  False  False  False     False  False

      who  adult_male   deck  embark_town  alive  alone
0    False      False   True       False  False  False
1    False      False  False       False  False  False
2    False      False   True       False  False  False
3    False      False  False       False  False  False
4    False      False   True       False  False  False
..     ...        ...    ...         ...    ...    ...
886  False      False   True       False  False  False
887  False      False  False       False  False  False
888  False      False   True       False  False  False
889  False      False  False       False  False  False
890  False      False   True       False  False  False

[891 rows x 15 columns]

data.isna().any():
survived       False
pclass         False
sex            False
age             True
sibsp          False
parch          False
fare           False
embarked        True
```

```
class          False
who            False
adult_male     False
deck           True
embark_town    True
alive          False
alone          False
dtype: bool

data.isna().sum():
survived       0
pclass         0
sex            0
age            177
sibsp          0
parch          0
fare           0
embarked       2
class          0
who            0
adult_male     0
deck           688
embark_town    2
alive          0
alone          0
dtype: int64

data.isna().any().sum():
4



---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[31], line 29
     26 print(data.isna().any().sum())
     28 #6.fillna()
---> 29 df_fill = data.fillna(10)
     30 df_fill = data.fillna({"age": 10})
     31 print("\nAfter fillna():\n", df_fill)

File ~\anaconda3\Lib\site-packages\pandas\core\generic.py:7159, in NDFrame.fillna(self, value,
axis, inplace, limit)
   7157         new_data = result._mgr
   7158     else:
-> 7159         new_data = self._mgr.fillna(value=value, limit=limit, inplace=inplace)
   7160 elif isinstance(value, ABCDataFrame) and self.ndim == 2:
   7161     new_data = self.where(self.notna(), value)._mgr

File ~\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:458, in
BaseBlockManager.fillna(self, value, limit, inplace)
```

```
    454 if limit is not None:
    455     # Do this validation even if we go through one of the no-op paths
    456     limit = libalgos.validate_limit(None, limit=limit)
--> 458 return self.apply(
    459         ,
    460     value=value,
    461     limit=limit,
    462     inplace=inplace,
    463 )

File ~\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:442, in
BaseBlockManager.apply(self, f, align_keys, **kwargs)
    440         applied = b.apply(f, **kwargs)
    441     else:
--> 442         applied = getattr(b, f)(**kwargs)
    443     result_blocks = extend_blocks(applied, result_blocks)
    445 out = type(self).from_blocks(result_blocks, [ax.view() for ax in self.axes])

File ~\anaconda3\Lib\site-packages\pandas\core\internals\blocks.py:1909, in
ExtensionBlock.fillna(self, value, limit, inplace)
   1906 except TypeError:
   1907     # 3rd party EA that has not implemented copy keyword yet
   1908     refs = None
-> 1909     new_values = self.values.fillna(value=value, limit=limit)
   1910     # issue the warning *after* retrying, in case the TypeError
   1911     #  was caused by an invalid fill_value
   1912     warnings.warn(
   1913         # GH#53278
   1914         "ExtensionArray.fillna added a 'copy' keyword in pandas "
  (...)   1920         stacklevel=find_stack_level(),
   1921     )

File ~\anaconda3\Lib\site-packages\pandas\core\arrays\_mixins.py:368, in
NDArrayBackedExtensionArray.fillna(self, value, limit, copy)
    366     else:
    367         new_values = self[:]
--> 368     new_values[mask] = value
    369 else:
    370     # We validate the fill_value even if there is nothing to fill
    371     self._validate_setitem_value(value)

File ~\anaconda3\Lib\site-packages\pandas\core\arrays\_mixins.py:272, in
NDArrayBackedExtensionArray.__setitem__(self, key, value)
    269     raise ValueError("Cannot modify read-only array")
    271 key = check_array_indexer(self, key)
--> 272 value = self._validate_setitem_value(value)
    273 self._ndarray[key] = value

File ~\anaconda3\Lib\site-packages\pandas\core\arrays\categorical.py:1653, in
Categorical._validate_setitem_value(self, value)
   1651     return self._validate_listlike(value)
```

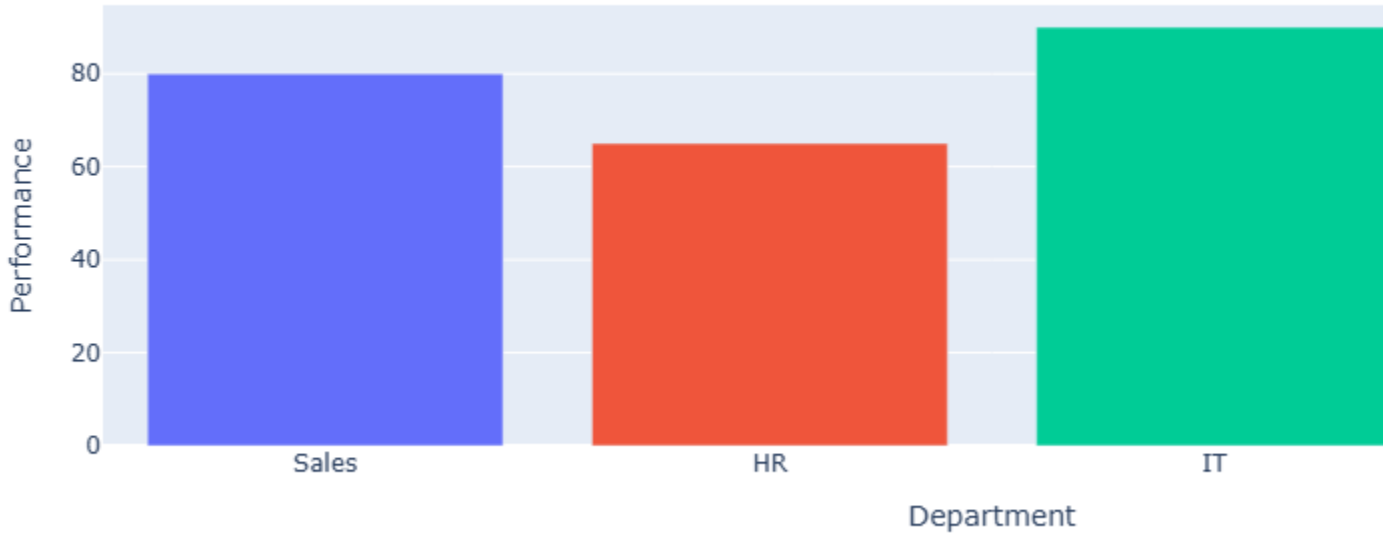
```
  1652 else:
-> 1653     return self._validate_scalar(value)

File ~\anaconda3\Lib\site-packages\pandas\core\arrays\categorical.py:1678, in
Categorical._validate_scalar(self, fill_value)
  1676     fill_value = self._unbox_scalar(fill_value)
  1677 else:
-> 1678     raise TypeError(
  1679         "Cannot setitem on a Categorical with a new "
  1680         f"category ({fill_value}), set the categories first"
  1681     ) from None
  1682 return fill_value

TypeError: Cannot setitem on a Categorical with a new category (10), set the categories first
```

In [37]:

```
#Visualizations that adapt based on time, location, or user role (e.g., admin vs. viewer).

import pandas as pd
import plotly.express as px
from datetime import datetime

# Sample dataset
data = {
    "Department": ["Sales", "HR", "IT", "Finance"],
    "Performance": [80, 65, 90, 75]
}

df = pd.DataFrame(data)

# User role input
role = input("Enter role (Admin / Viewer): ")

# Detect current time
current_hour = datetime.now().hour

print("Current Hour:", current_hour)

# Create visualization
fig = px.bar(df, x="Department", y="Performance", color="Department")

# Role-based visualization
if role.lower() == "admin":
    print("Admin Access: Full Data")
    print(df)
    fig.show()
```

```
elif role.lower() == "viewer":
    print("Viewer Access: Chart Only")
    fig.show()

else:
    print("Invalid Role")

# Time-based message
if current_hour < 12:
    print("Morning Dashboard View")
else:
    print("Evening Dashboard View")
```

Current Hour: 15
Viewer Access: Chart Only



Evening Dashboard View

In [36]:

```
#Visualizations that adapt based on time, location, or user role (e.g., admin vs. viewer).

#import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from datetime import datetime

# Load seaborn dataset
df = sns.load_dataset("tips")

# User role input
role = input("Enter role (Admin / Viewer): ")

# Detect current time
current_hour = datetime.now().hour

print("Current Hour:", current_hour)

# Create visualization
plt.figure(figsize=(6,4))
sns.barplot(x="day", y="total_bill", data=df, palette="Set2",legend=False)
plt.title("Average Total Bill by Day")

# Role-based visualization
if role.lower() == "admin":
    print("Admin Access: Full Data")
    print(df.head())
    plt.show()
elif role.lower() == "viewer":
    print("Viewer Access: Chart Only")
    plt.show()
else:
    print("Invalid Role")

# Time-based message
if current_hour < 12:
    print("Morning Dashboard View")
else:
    print("Evening Dashboard View")
```

```
Current Hour: 15
```

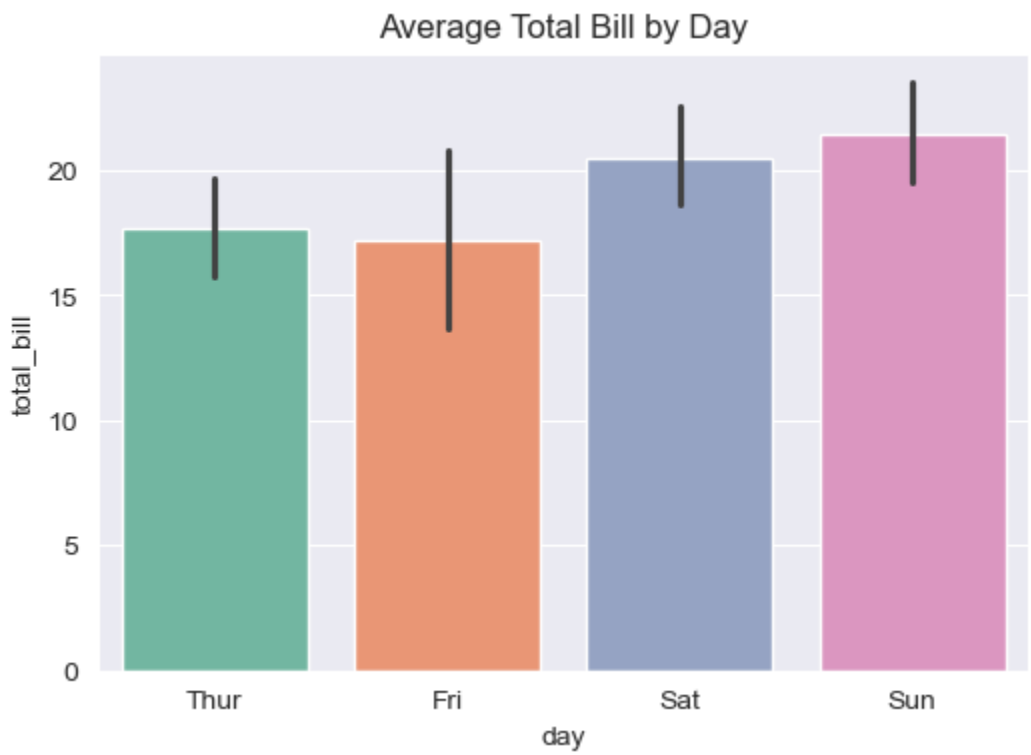
C:\Users\Poonam\AppData\Local\Temp\ipykernel_9608\3557080490.py:21: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x="day", y="total_bill", data=df, palette="Set2",legend=False)
```

Admin Access: Full Data

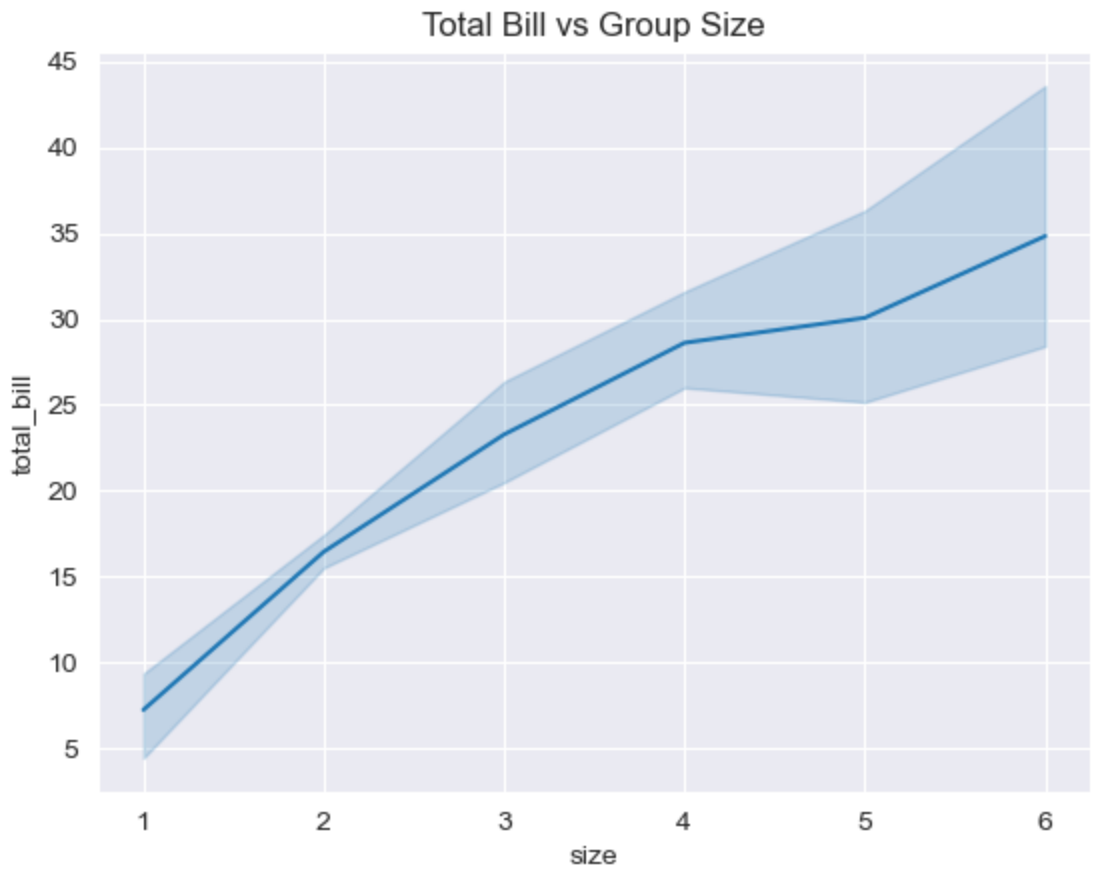
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4



Evening Dashboard View

In [34]:

```
#Use user profiles or preferences to customize data representation.

import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = sns.load_dataset("tips")

# User preference
chart_type = input("Enter preferred chart (bar / scatter / line / box / histogram): ")
# Visualization based on user preference
#Barplot
if chart_type.lower() == "bar":
    sns.barplot(x="day", y="total_bill", data=df)
    plt.title("Average Total Bill by Day")
#Scatter plot
elif chart_type.lower() == "scatter":
    sns.scatterplot(x="total_bill", y="tip", data=df)
    plt.title("Total Bill vs Tip")
#line plot
elif chart_type.lower() == "line":
    sns.lineplot(x="size", y="total_bill", data=df)
    plt.title("Total Bill vs Group Size")
#box plot
elif chart_type.lower() == "box":
    sns.boxplot(x="day", y="total_bill", data=df)
    plt.title("Bill Distribution by Day")
#Histogram plot
elif chart_type.lower() == "histogram":
    sns.histplot(df["total_bill"], bins=10)
    plt.title("Distribution of Total Bill")
else:
    print("Invalid chart type")
plt.show()
```

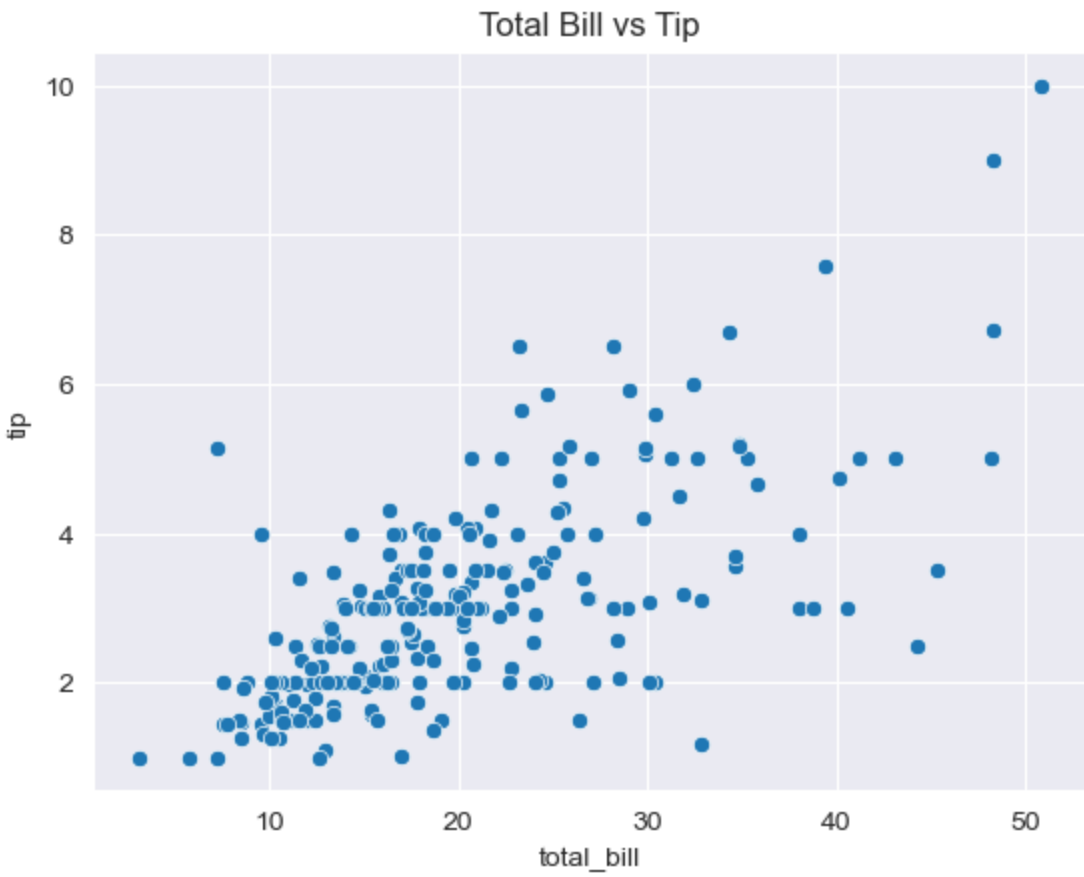


In [33]:

```
#Create narrative visualizations that change dynamically based on user behavior.

import seaborn as sns
import matplotlib.pyplot as plt

# Load dataset
df = sns.load_dataset("tips")

# User behavior input
choice = input("Select view (overview / relationship / distribution): ")

# Narrative visualization
if choice.lower() == "overview":

    print("Narrative: This chart shows the average total bill across different days.")
    sns.barplot(x="day", y="total_bill", data=df)
    plt.title("Average Total Bill by Day")
```

```
elif choice.lower() == "relationship":

    print("Narrative: This plot shows how tips vary with total bill amount.")
    sns.scatterplot(x="total_bill", y="tip", data=df)
    plt.title("Total Bill vs Tip")

elif choice.lower() == "distribution":

    print("Narrative: This histogram shows the distribution of total bill values.")
    sns.histplot(df["total_bill"], bins=10)
    plt.title("Distribution of Total Bill")

else:
    print("Invalid choice")

plt.show()
```

Narrative: This plot shows how tips vary with total bill amount.



In [ ]: